République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université de Tunis El Manar
École Nationale d'Ingénieurs de Tunis
Département Génie Électrique

PROJET FIN D'ANNEE II

Commande Neuronale d'un Système Thermique à Distance

Réalisé par :

Ouri Mohamed Amine
Chaanbi Hassen

2ème Année GE 2

Encadré par : Mr. Bouani Faouzi

Année universitaire : 2025/2026

Signatures & Approbations

Tunis, _____ 2026

Remerciements

Au terme de ce travail, nous tenons à exprimer notre profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'année.

Nous adressons nos plus sincères remerciements à notre encadrant, **Monsieur Faouzi Bouani**, Professeur à l'École Nationale d'Ingénieurs de Tunis, pour la confiance qu'il nous a accordée en nous proposant ce sujet, pour sa disponibilité constante, ses conseils avisés et son suivi rigoureux tout au long de ce travail. Sa pédagogie et son expertise dans le domaine de la commande des systèmes ont été une source d'inspiration et d'orientation précieuse pour l'aboutissement de ce projet.

Nous exprimons également notre reconnaissance à l'ensemble du corps enseignant et administratif de notre établissement pour la qualité de la formation dispensée tout au long de notre cursus.

Enfin, nous tenons à remercier chaleureusement nos familles et nos proches pour leur soutien moral et leurs encouragements continus qui nous ont permis de mener ce travail dans les meilleures conditions.

Chaanbi Hassen & Ouri Mohamed Amine

Résumé

Ce rapport s'inscrit dans le cadre d'un Projet de Fin d'Année (PFA) du deuxième année génie électrique à l'école nationale d'ingénieurs de Tunis ,et porte sur la conception d'un régulateur pour un système thermique basé sur les réseaux de neurones artificiels. Le projet articule deux volets complémentaires : d'une part, une présentation rigoureuse des fondements mathématiques sous-jacents à cette approche de commande intelligente, et d'autre part, une étude comparative mettant en évidence les performances et l'efficacité du régulateur neuronal par rapport aux méthodes de régulation classiques.

Abstract

This report is part of a final-year project and focuses on the design of a controller for a thermal system based on artificial neural networks. The project comprises two complementary parts : firstly, a rigorous presentation of the mathematical foundations underlying this intelligent control approach, and secondly, a comparative study highlighting the performance and efficiency of the neural controller compared to classical control methods.

Table des matières

Remerciements ii

Résumé iii

Table des figures vi

Liste des tableaux vii

Introduction Générale 1

1 Fondements théoriques :systémes et réseaux de neurones 3
- 1.1 Généralités . . . 3
  - 1.1.1 Représentation des systèmes . . . 3
  - 1.1.2 Modèle d'un système . . . 4
- 1.2 Les réseaux de neurones artificiels . . . 4
  - 1.2.1 Historique des réseaux de neurones artificiels . . . 4
  - 1.2.2 Commande neuronale d'un système inconnu . . . 6
  - 1.2.3 Principes du MLP . . . 6
  - 1.2.4 L'aspect mathématique du neurone artificiel . . . 7
  - 1.2.5 Fonctions d'activation . . . 7
  - 1.2.6 Architecture du Perceptron Multi-Couche (MLP) . . . 9
  - 1.2.7 Apprentissage par rétropropagation du gradient . . . 10
  - 1.2.8 Le modèle récursif général du MLP . . . 12

2 Validation par simulation MATLAB : PID, identification et commande MVC 14
- 2.1 Simulation et analyse de la commande PID numérique . . . 15
  - 2.1.1 Commande PID d'un système du second ordre . . . 15
    - 2.1.1.1 Présentation du système et discrétisation . . . 15
    - 2.1.1.2 Loi de commande PID discrète et calcul des coefficients . . . 16
    - 2.1.1.3 Résultats de simulation et interprétation . . . 16
- 2.2 Identification par réseaux de neurones . . . 17
  - 2.2.1 Objectif et démarche . . . 17
  - 2.2.2 Identification du système non linéaire à mémoire . . . 17
    - 2.2.2.1 Propriétés du système . . . 17
    - 2.2.2.2 Topologie du neurone d'identification . . . 18
    - 2.2.2.3 Résultats et interprétation . . . 19

2.3 Commande par minimum de variance 20
2.3.1 Critère de performance 20
2.3.2 Algorithme d'optimisation itérative 20
2.3.3 Résultats et analyse du suivi 21

3 Implémentation pratique 23
3.1 Introduction 23
3.2 Architecture matérielle 23
3.2.1 Carte de développement ESP32 23
3.2.2 AC Dimmer Module 24
3.2.3 Capteur de température DHT22 25
3.2.4 Equipement de chauffage 220V 26
3.3 Architecture du système 26
3.3.1 Essai en boucle ouverte 26
3.4 Apprentissage du neurone artificiel 27
3.5 Essai en boucle fermée 30
3.5.1 Implémentation de l'algorithme sur la carte 30
3.5.2 Montage expérimental 31
3.5.3 Déroulement de l'essai 31
3.5.4 Résultats expérimentaux et analyse 32
3.6 Communication sans fil 33
3.6.1 Le protocole ESP-NOW 33
3.6.2 Implémentation des algorithmes de commandes sur les microcontroleurs ESP32 . . 33
3.6.3 Montage expérimental 34
3.6.4 Résultats expérimentaux et analyse 34

Conclusion générale 37

Bibliographie 38

Table des figures

1.1 Représentation des systèmes monovariable et multivariable. 4
1.2 Structure de commande neuronale en boucle fermée. Le MLP joue le rôle du contrôleur neuronal direct. 6
1.3 Schéma structurel d'un neurone artificiel. 7
1.4 Fonction d'activation Heaviside. 8
1.5 Graphique de la fonction d'activation sigmoïde. 8
1.6 Graphique de la fonction d'activation tangente hyperbolique. 9
1.7 Architecture d'un MLP à $L = 4$ couches cachées ($n_0 = 3,\ n_1 = 4,\ n_2 = 4,\ n_3 = 2,\ n_4 = 1$). 10

2.1 Réponse indicielle du système du second ordre en boucle ouverte. 15
2.2 Simulation de la commande PID appliquée au système du second ordre ($K_s = 5$, $\omega_0 = 20\,\text{rad/s}$, $\xi = 0{,}5$). 16
2.3 Évolution de la commande aléatoire $u(k) \in [-0{,}5,\ 0{,}5]$. 19
2.4 Résultats de l'identification du système à mémoire. 19
2.5 Résultats de la commande par minimum de variance. 21
2.6 Changement de la réactivité du système pour les grandes valeurs de régularisation sur la commande. 22

3.1 Le microcontroleur ESP32 23
3.2 Module actionneur à Triacc (Dimmer) 25
3.3 Capteur d'humidité et de température DHT22 25
3.4 Schéma synoptique du système de régulation thermique 26
3.5 Evolution de la température en fonction du temps 27
3.6 Évolution de la commande en puissance pendant l'essai en boucle ouverte 28
3.7 Évolution de la sortie système et de la sortie prédite du neurone pendant l'apprentissage . 29
3.8 Évolution de la sortie système et de la sortie prédite du neurone pendant le test 30
3.9 Montage expérimental 31
3.10 Évolution de la sortie, de la consigne et de la commande pour l'essai en boucle fermée . . 32
3.11 Schéma de communication entre les deux ESP32 33
3.12 Montage expérimental avec communication 34
3.13 Évolution de la sortie, de la consigne et de la commande 35

Liste des tableaux

2.1 Paramètres du système du second ordre. 15
2.2 Paramètres du correcteur PID discret pour le système du second ordre. 16
2.3 Effet du pas d'adaptation μ sur la convergence. 18
2.4 Bilan comparatif de l'identification neuronale. 20

3.1 Extraits du datasheet du microcontroleur ESP32 24
3.2 Caractéristiques du capteur DHT22 25
3.3 Caractéristiques du protocole ESP-NOW 33

Introduction Générale

L'évolution technologique actuelle se trouve à la croisée de l'intelligence artificielle et du génie automatique. Les réseaux de neurones artificiels sont une technologie de rupture au cœur de cette synergie. En s'inspirant des mécanismes biologiques du cerveau humain, les réseaux de neurones artificiels donnent aux machines la capacité d'apprendre et de s'adapter, ce qui pourrait transformer profondément la gestion des systèmes complexes.

Dans le domaine de la commande de systèmes, les approches classiques, comme les régulateurs PID ou la commande prédictive, ont fait leurs preuves sur les systèmes linéaires dont le modèle mathématique est parfaitement connu. Ces méthodes ont toutefois leurs limites lorsqu'elles sont appliquées à des systèmes non linéaires, incertains ou soumis à des perturbations environnementales imprévisibles. C'est ici que la commande neuronale montre tout son intérêt : elle propose une alternative puissante, capable d'approximer des fonctions complexes et d'apprendre la dynamique d'un système à partir de données réelles, sans qu'il soit besoin de modélisation mathématique explicite.

L'aptitude à s'adapter est encore plus critique si l'on considère qu'une grande part des systèmes réels sont exposés à des facteurs exogènes aléatoires et non prévues . De même que la littérature récente concernant la prédiction de la consommation énergétique, le réseau neuronal est capable de détecter des corrélations non-linéaires dans des ensembles de données perturbées, alors que l'approche analytique traditionnelle aurait échoué faute de paramètres.

L'objectif de ce projet est de concevoir et mettre en œuvre un régulateur fondé sur les réseaux de neurones, en suivant une démarche rigoureuse allant de la théorie jusqu'à la validation expérimentale. Ce rapport est structuré en trois chapitres complémentaires :

- **Chapitre 1 : Fondements théoriques et évolution des commandes.** Ce chapitre expose les bases conceptuelles du projet. Nous y décrivons l'évolution de l'automatique, depuis la formalisation des régulateurs PID classiques jusqu'à l'émergence des architectures neuronales. Cette étude comparative montre les avantages des méthodes conventionnelles tout en pointant les limites structurelles qui rendent légitime le recours à des approches bio-inspirées plus avancées.
- **Chapitre 2 : Étude comparative par simulation numérique.** Consacré au cœur applicatif sous environnement MATLAB, ce chapitre se déroule suivant un ordre méthodique. Nous commençons par y évaluer les performances des régulateurs PID sur des modèles du second ordre. Nous mettons ensuite en œuvre et simulons des architectures neuronales (MLP et réseaux récurrents) pour des signaux non linéaires et des systèmes à mémoire, afin de valider la supériorité de l'apprentissage par descente de gradient dans ce contexte de non-linéarité.
- **Chapitre 3 : Mise en œuvre expérimentale et validation physique.** C'est cette partie qui

passe de la simulation à la réalité. C'est à partir d'elle qu'on s'intéresse à l'utilisation concrète du régulateur dans un système thermique réel (enclosure fermée contrôlé par un ESP32). On s'y penchera en particulier sur l'instrumentation, la mise en place de la base de données expérimentale et la comparaison des capacités du régulateur neuronal face aux contraintes réelles (mesure aléatoire et inertie thermique).

Chapitre 1

Fondements théoriques :systémes et réseaux de neurones

Introduction

Ce chapitre présente les principes théoriques et conceptuels nécessaires pour comprendre ce travail. Il est divisé en deux parties complémentaires. La première partie concerne les principes généraux de la commande des systèmes. La deuxième partie concerne les réseaux de neurones artificiels et leur application à la commande des systèmes non linéaires. L'objectif principal de ce chapitre est de justifier le choix d'une approche neuronale pour la régulation thermique d'une enceinte, lorsque le modèle mathématique du système est inconnu ou difficile à identifier.

1.1 Généralités

L'automatique est la discipline de l'ingénieur qui répond à une question fondamentale : comment faire en sorte qu'un système se comporte comme on le souhaite ? Qu'il s'agisse de maintenir la température d'un four, de stabiliser un drone en vol, ou de réguler la vitesse d'un moteur, le problème est toujours le même : il faut calculer une action de commande appropriée, en temps réel, malgré les perturbations et les incertitudes.

La commande classique repose sur la connaissance d'un modèle mathématique du système. Ce modèle mathématique peut être une équation différentielle ou une fonction de transfert. Il décrit la relation entre l'entrée de commande, notée $u(t)$, et la sortie, notée $y(t)$. Une fois que ce modèle mathématique est identifié, il permet de prédire le comportement du système. Il permet également de concevoir une loi de commande adaptée au système.

L'outil central de cette démarche est le schéma fonctionnel, également appelé schéma-blocs. Le schéma fonctionnel représente graphiquement les interactions entre les différents composants du système.

1.1.1 Représentation des systèmes

On distingue deux grandes classes de systèmes selon le nombre de leurs variables d'entrée et de sortie.

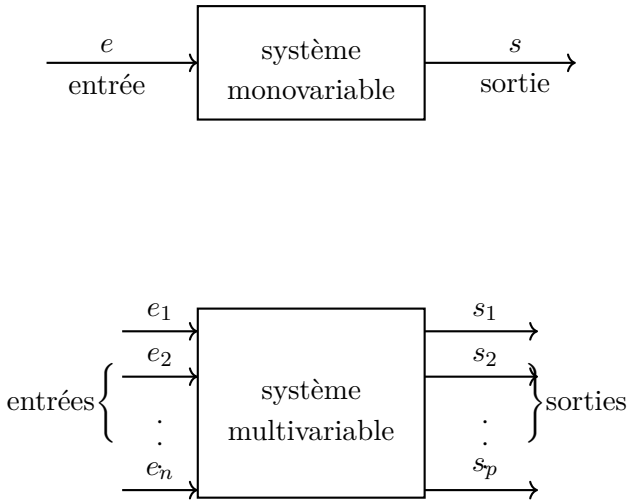


FIGURE 1.1 – Représentation des systèmes monovariable et multivariable.

Un **système monovariable** (SISO) possède une seule entrée et une seule sortie, ce qui est le cas de l'enceinte thermique étudiée dans ce travail : l'entrée est la puissance de chauffe $u(t)$ et la sortie est la température $y(t)$. Un **système multivariable** (MIMO) possède plusieurs entrées et plusieurs sorties, ce qui complique significativement la synthèse de la loi de commande.

1.1.2 Modèle d'un système

Pour concevoir une loi de commande, il est indispensable de disposer d'une représentation mathématique du système. On distingue deux approches :

- **Modèle de connaissance** : construit à partir des lois physiques (thermodynamique, mécanique, électromagnétisme). Il est précis mais souvent trop complexe pour être utilisé directement en commande, car il peut impliquer des équations aux dérivées partielles ou des paramètres difficiles à mesurer.
- **Modèle de commande** (ou modèle boîte noire) : obtenu à partir de mesures expérimentales grâce à une procédure d'identification. Les paramètres physiques n'apparaissent pas explicitement. Seule la relation entrée-sortie est modélisée. C'est cette approche qui est retenue dans ce travail.

1.2 Les réseaux de neurones artificiels

1.2.1 Historique des réseaux de neurones artificiels

L'histoire des réseaux de neurones artificiels est marquée par des avancées théoriques majeures.

- **McCulloch et Pitts (1943)** e premier modèle mathématique du neurone biologique est celui du neurone formel [**?**]. Ce modèle produit une sortie binaire. Cette sortie binaire est obtenue par seuillage d'une somme pondérée.

$$y \;=\; \mathbf{1}_{\left\{\sum_{i=1}^{n} w_i\, x_i \,\geq\, \theta\right\}}$$

Ce modèle montre qu'un assemblage de neurones de cette façon peut réaliser n'importe quelle fonction logique.

- **Règle de Hebb (1949)** D. Hebb introduit le premier mécanisme d'apprentissage : deux neurones activés simultanément renforcent leur connexion synaptique :

$$\Delta w_{ij} \;=\; \eta\, a_i\, a_j$$

C'est le premier exemple d'apprentissage non supervisé.

- **Perceptron et Adaline (1958)** À cette époque, la notion du perceptron a été introduite. Le perceptron est le premier réseau à apprentissage supervisé. La règle de correction des poids du perceptron est proportionnelle à l'erreur de prédiction du perceptron.

$$w_i \;=\; w_i + \eta\,(y - \hat{y})\,x_i$$

Widrow et Hoff ont proposé en même temps l'**Adaline**. L'Adaline minimise l'erreur quadratique, notée $\mathcal{L}$, qui est égale à $\frac{1}{2}(y-\hat{y})^2$. Ceci est réalisé par la méthode de descente de gradient. L'Adaline constitue le modèle de base des réseaux multicouches.

- **Limites du Perceptron (Minsky et Papert, 1969)** Démonstration rigoureuse que le Perceptron ne résout que des problèmes linéairement séparables. En effet, il ne peut pas réaliser la fonction XOR :

$$\nexists\,(w_1, w_2, \theta) \;\text{ tels que }\; w_1x_1 + w_2x_2 \geq \theta \iff x_1 \oplus x_2 = 1$$

Cette démonstration provoque un premier hiver des réseaux de neurones.

- **Cartes de Kohonen et machine de Boltzmann (1985)** Kohonen propose les cartes auto-organisatrices (apprentissage non supervisé). Hinton introduit la machine de Boltzmann, dont les états suivent une distribution de Gibbs-Boltzmann :

$$P(\mathbf{s}) \;=\; \frac{1}{Z}\, e^{-E(\mathbf{s})/T}$$

- **1986 — MLP et rétropropagation du gradient** Rumelhart, Hinton et Williams formalisent l'algorithme de **rétropropagation**, qui permet d'entraîner des réseaux multicouches. La propagation avant calcule :

$$\mathbf{a}^{(\ell)} \;=\; f\Big(\mathbf{W}^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}\Big)$$

et la mise à jour des poids s'effectue par :

$$\mathbf{W}^{(\ell)} \leftarrow \mathbf{W}^{(\ell)} - \eta\, \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}}$$

Le **théorème d'approximation universel** (Cybenko, 1989) établit qu'un MLP à une couche cachée peut approximer toute fonction continue avec une précision arbitraire.

- **Jusqu'à aujourd'hui** Les réseaux de neurones profonds forment la base de l'intelligence artificielle

moderne. Ils ont de nombreuses applications, notamment la commande de systèmes, le traitement du signal, la vision par ordinateur et l'analyse de séries temporelles. Les réseaux de neurones profonds sont vraiment très importants pour l'intelligence artificielle moderne.

1.2.2 Commande neuronale d'un système inconnu

Soit un système (S) à entrée $u(t)$ et sortie $y(t)$ dont le modèle mathématique est **inconnu**. L'objectif est de générer $u(t)$ telle que $y(t)$ suive une consigne $r(t)$:

$$e(t) \;=\; r(t) - y(t) \;\longrightarrow\; 0$$

- **Approche classique** : un régulateur PID nécessite la connaissance explicite du modèle. Sans modèle, cette approche est inapplicable.
- **Approche neuronale** : le MLP joue deux rôles complémentaires.
  - **Identification** : un premier MLP apprend le comportement entrée-sortie du système :

$$\hat{y}(t) = f_{\text{MLP}}(u(t),\, u(t-1),\, y(t-1),\, \ldots)$$

  - **Commande** : un second MLP génère la commande en minimisant :

$$\mathcal{L} \;=\; \frac{1}{N}\sum_{k=1}^{N}\Big(r(t_k) - y(t_k)\Big)^2$$

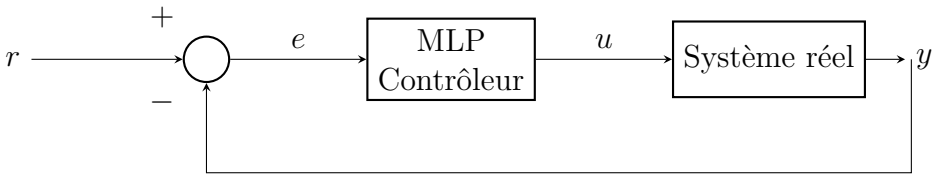


FIGURE 1.2 – Structure de commande neuronale en boucle fermée. Le MLP joue le rôle du contrôleur neuronal direct.

La puissance de cette approche réside dans le fait que le MLP apprend **implicitement** la dynamique du système à partir des données d'entrée-sortie, sans modèle analytique explicite.

1.2.3 Principes du MLP

L'implémentation des réseaux de neurones artificiels repose sur deux principes :

- **La mémoire informatique** :
  Les paramètres du réseau ($\mathbf{W}^{(\ell)}$ et $\mathbf{b}^{(\ell)}$) sont stockés sous forme de tableaux de nombres réels. À chaque époque d'apprentissage, ces valeurs sont mises à jour et conservées. Cela permet l'accumulation progressive de l'information au fil du temps.

- **La méthode de résolution de l'analyse numérique** : La rétropropagation dans le calcul des paramètres du réseau implique des produits matriciels, l'évaluation de fonctions d'activation et le calcul de gradients. Tous ces éléments font partie de l'analyse numérique itérative.

1.2.4 L'aspect mathématique du neurone artificiel

Un neurone artificiel reçoit des entrées, qui sont notées $x_1, x_2, \ldots, x_n$. Chaque entrée a un poids synaptique, qui est noté w_i. Le neurone produit ensuite une sortie, qui est un nombre appelé y, et cela se fait en trois étapes.

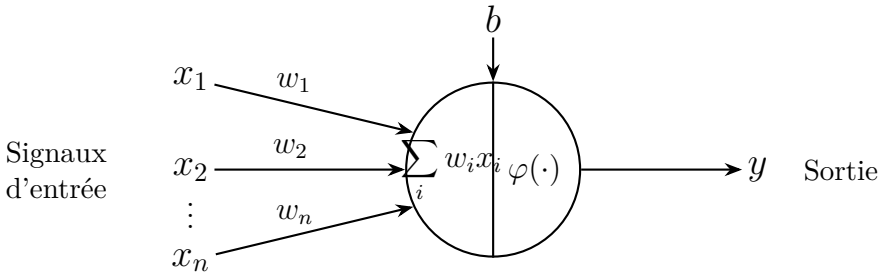


FIGURE 1.3 – Schéma structurel d'un neurone artificiel.

1. Pondération et sommation.

$$v \;=\; \sum_{i=0}^{n} w_i\, x_i \;=\; \mathbf{w}^\top \mathbf{x} \tag{1.2.1}$$

où $x_0 = 1$ et $w_0 = b$ (biais).

2. Activation.

$$y \;=\; f(v) \;=\; f\Big(\mathbf{w}^\top \mathbf{x}\Big) \tag{1.2.2}$$

1.2.5 Fonctions d'activation

Fonction échelon (Heaviside).

$$f(v) \;=\; \begin{cases} 1 & v \geq \theta \\ 0 & \text{sinon} \end{cases} \tag{1.2.3}$$

Non dérivable, elle est inadaptée à l'apprentissage par gradient.

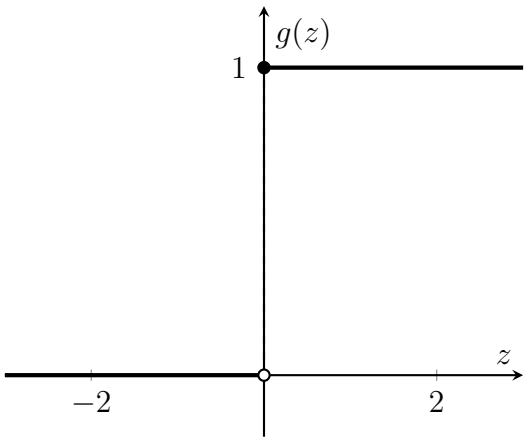


FIGURE 1.4 – Fonction d'activation Heaviside.

Fonction linéaire.

$$f(v) \;=\; \alpha\, v, \qquad \alpha \in \mathbb{R}^{+} \tag{1.2.4}$$

Utilisée en couche de sortie pour les problèmes de régression.

Fonction sigmoïde.

$$f(v) \;=\; \sigma(v) \;=\; \frac{1}{1+e^{-v}}, \qquad f(v) \in [0,\, 1] \tag{1.2.5}$$

Sa dérivée est :

$$\sigma'(v) \;=\; \sigma(v)\left[1-\sigma(v)\right] \tag{1.2.6}$$

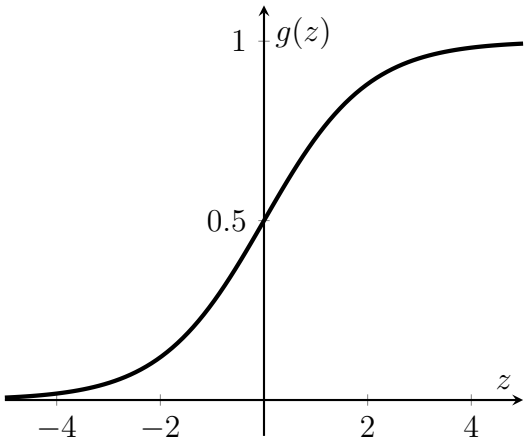


FIGURE 1.5 – Graphique de la fonction d'activation sigmoïde.

Fonction tangente hyperbolique.

$$f(v) \;=\; \tanh(v) \;=\; \frac{e^{v}-e^{-v}}{e^{v}+e^{-v}}, \qquad f(v) \in (-1,\, 1) \tag{1.2.7}$$

Sa dérivée est :

$$\tanh'(v) \;=\; 1 - \tanh^2(v) \tag{1.2.8}$$

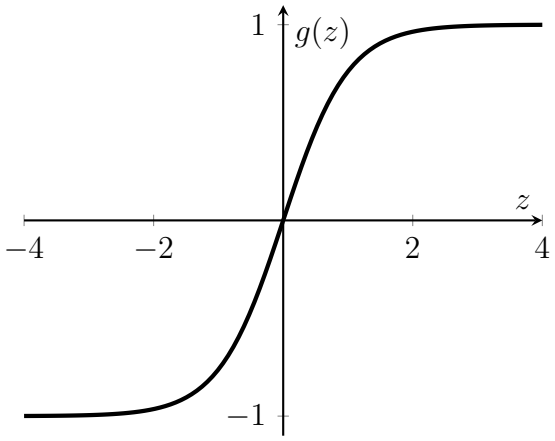


FIGURE 1.6 – Graphique de la fonction d'activation tangente hyperbolique.

1.2.6 Architecture du Perceptron Multi-Couche (MLP)

Un MLP à L couches cachées est constitué d'une couche d'entrée, de L couches cachées et d'une couche de sortie.

Considérons un réseau à $L = 4$ couches cachées, avec par exemple $n_0 = 3$, $n_1 = 4$, $n_2 = 4$, $n_3 = 2$, $n_4 = 1$ neurones respectivement (soit 5 couches au total avec la couche d'entrée).

— n_ℓ : nombre de neurones de la couche ℓ. Pour $\ell = 1$, on a $n_1 = 4$ neurones dans la première couche cachée.

— $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$: matrice des poids de la couche ℓ. Pour $\ell = 1$, elle relie les $n_0 = 3$ entrées aux $n_1 = 4$ neurones :

$$\mathbf{W}^{(1)} \in \mathbb{R}^{4\times 3} \;=\; \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} & w_{33}^{(1)} \\ w_{41}^{(1)} & w_{42}^{(1)} & w_{43}^{(1)} \end{pmatrix}$$

— $\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_\ell}$: vecteur des biais de la couche ℓ. Pour $\ell = 1$:

$$\mathbf{b}^{(1)} = \left[b_1^{(1)},\; b_2^{(1)},\; b_3^{(1)},\; b_4^{(1)}\right]^\top \in \mathbb{R}^4$$

— $\mathbf{a}^{(\ell)} \in \mathbb{R}^{n_\ell}$: vecteur des activations de la couche ℓ. Pour $\ell = 1$:

$$\mathbf{a}^{(1)} = \left[a_1^{(1)},\; a_2^{(1)},\; a_3^{(1)},\; a_4^{(1)}\right]^\top \in \mathbb{R}^4$$

et la relation de propagation s'écrit :

$$\mathbf{a}^{(\ell)} = \varphi\Big(\mathbf{W}^{(\ell)}\,\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}\Big), \quad \ell = 1, 2, 3, 4$$

avec $\mathbf{a}^{(0)} = \mathbf{x}$ le vecteur d'entrée.

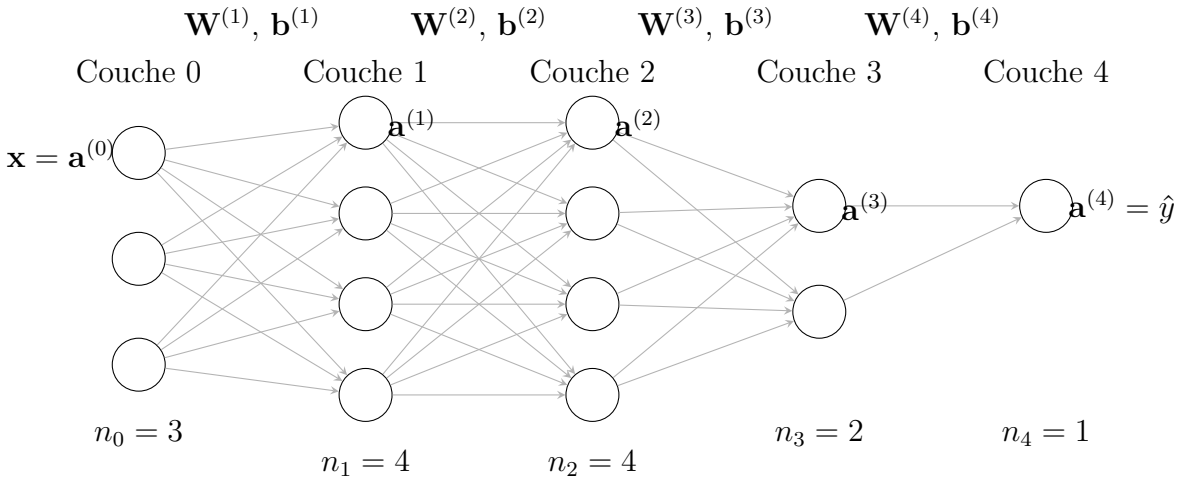


FIGURE 1.7 – Architecture d'un MLP à $L = 4$ couches cachées ($n_0 = 3,\ n_1 = 4,\ n_2 = 4,\ n_3 = 2,\ n_4 = 1$).

Propagation avant (*forward pass*). Pour $\ell = 1, \ldots, L$:

$$\mathbf{z}^{(\ell)} \;=\; \mathbf{W}^{(\ell)}\,\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)} \tag{1.2.9}$$

$$\mathbf{a}^{(\ell)} \;=\; f^{(\ell)}\Big(\mathbf{z}^{(\ell)}\Big) \tag{1.2.10}$$

avec $\mathbf{a}^{(0)} = \mathbf{x}$ et $\hat{\mathbf{y}} = \mathbf{a}^{(L)}$.

Réseaux de neurones à une couche cachée ($L = 2$ couches cachées au sens strict, mais ici on note $L = 3$ couches totales) Pour n entrées du réseau, q neurones de la couche cachée et une seule sortie :

$$\hat{y} \;=\; \sum_{j=1}^{q} w_j^{(2)}\,\tanh\left(b_j^{(1)} + \sum_{i=1}^{n} w_{ji}^{(1)}\,x_i\right) + b^{(2)} \tag{1.2.11}$$

1.2.7 Apprentissage par rétropropagation du gradient

Fonction de coût et problème de minimisation

L'objectif de l'apprentissage est de trouver les paramètres optimaux $\boldsymbol{\theta} = \{\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}\}$ qui minimisent l'erreur quadratique moyenne :

$$\mathcal{L}(\boldsymbol{\theta}) \;=\; \frac{1}{N}\sum_{k=1}^{N}\Big\|\mathbf{y}^{(k)} - \hat{\mathbf{y}}^{(k)}\Big\|^2 \tag{1.2.12}$$

où $\hat{\mathbf{y}}^{(k)}$ est la sortie du réseau pour le k-ième exemple, obtenue par propagation avant successive :

$$\hat{\mathbf{y}}^{(k)} = \mathbf{a}^{(L,k)}, \qquad \mathbf{a}^{(\ell,k)} = \varphi\Big(\mathbf{W}^{(\ell)}\,\mathbf{a}^{(\ell-1,k)} + \mathbf{b}^{(\ell)}\Big), \quad \ell = 1, \ldots, L \tag{1.2.13}$$

avec $\mathbf{a}^{(0,k)} = \mathbf{x}^{(k)}$ le vecteur d'entrée, soit trouver :

$$\boldsymbol{\theta}^* \;=\; \arg\min_{\boldsymbol{\theta}} \; \mathcal{L}(\boldsymbol{\theta})$$

Pourquoi la descente de gradient ?

Il y a deux méthodes de résolution mathématique pour minimiser $\mathcal{L}(\boldsymbol{\theta})$:

- **Solution analytique** : celle de l'algèbre linéaire qui consiste à annuler les dérivées partielles $\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} = 0$ conduit à l'inversion d'une matrice de taille $D \times D$, de complexité $\mathcal{O}(D^3)$. Pour un réseau avec des milliers de paramètres, cette opération est **numériquement prohibitive**.
- **Descente de gradient de l'analyse numérique** : qui consiste à corriger itérativement les paramètres du réseau dans le sens opposé au gradient :

$$\boldsymbol{\theta} \;\leftarrow\; \boldsymbol{\theta} - \eta\,\nabla_{\boldsymbol{\theta}}\,\mathcal{L}$$

  La complexité de chaque itération est $\mathcal{O}(ND)$. Puisque $\mathcal{L}$ est convexe, la convergence vers le minimum global est garantie.

Algorithme de rétropropagation

Dans le cas d'un neurone unique ($L = 1$, $n_0 = 3$, $n_1 = 1$), le réseau se réduit à :

$$\hat{y} = \varphi(w_1 x_1 + w_2 x_2 + w_3 x_3 + b)$$

où $w_1, w_2, w_3 \in \mathbb{R}$ sont les poids et $b \in \mathbb{R}$ le biais.

Étape 1 : Signal d'erreur en sortie.

$$\delta = \Big(\hat{y} - y\Big) \cdot \varphi'(z) \;\in\; \mathbb{R} \tag{1.2.14}$$

où $z = w_1 x_1 + w_2 x_2 + w_3 x_3 + b \in \mathbb{R}$.

Étape 2 : Mise à jour des paramètres.

$$
\begin{aligned}
w_1 &\leftarrow w_1 - \eta\, \delta\, x_1 \\
w_2 &\leftarrow w_2 - \eta\, \delta\, x_2 \\
w_3 &\leftarrow w_3 - \eta\, \delta\, x_3 \\
b &\leftarrow b - \eta\, \delta
\end{aligned}
$$

Ces deux étapes sont répétées sur l'ensemble des données jusqu'à convergence de $\mathcal{L}$ vers son minimum.

1.2.8 Le modèle récursif général du MLP

De façon générale, un MLP à $L+2$ couches peut s'écrire récursivement :

$$
x_{\ell+1} \;=\; \begin{cases} g(\theta_\ell\, x_\ell) & \ell = 0, 1, \ldots, L-1 \\ f(\theta_L\, x_L) & \ell = L \end{cases}
$$

où $\theta_\ell = (b_\ell \mid W_\ell)$ est la matrice de poids partitionnée ($W_\ell \in \mathbb{R}^{n_{\ell+1} \times n_\ell}$, $b_\ell \in \mathbb{R}^{n_{\ell+1}}$), x_0 est le vecteur d'entrée, x_{L+1} la sortie, et g, f sont respectivement les fonctions d'activation des couches cachées et de la couche de sortie.

Conclusion

Ce premier chapitre a présenté les éléments essentiels pour comprendre le travail. Les principes de base de la commande en boucle fermée ont été définis, en incluant la définition des critères de performance importants — la précision, la rapidité et la stabilité — qui sont les principaux défis à relever lors de la création d'un régulateur.

Le régulateur PID, dont la loi de commande :

$$
u(t) \;=\; K_p\, e(t) + K_i \int_0^t e(\tau)\, \mathrm{d}\tau + K_d\, \frac{\mathrm{d}e(t)}{\mathrm{d}t}
$$

offre un compromis éprouvé depuis plus d'un siècle. Cependant, ses performances sont conditionnées par la connaissance du modèle mathématique du système, ce qui constitue une limitation majeure face aux systèmes non linéaires ou mal connus.

C'est pour répondre à cette limitation que les réseaux de neurones artificiels ont été introduits. Le MLP, dont l'apprentissage par rétropropagation minimise :

$$
\mathcal{L}(\boldsymbol{\theta}) \;=\; \frac{1}{N} \sum_{k=1}^{N} \left\| \mathbf{y}^{(k)} - \hat{\mathbf{y}}^{(k)} \right\|^2
$$

est capable d'identifier implicitement la dynamique d'un système inconnu à partir de ses seules données

d'entrée-sortie.

Fort de ces bases, le chapitre suivant présente les simulations réalisées pour valider ces deux approches sur des systèmes linéaires et non linéaires, avant leur application expérimentale à la régulation thermique de l'enceinte.

Chapitre 2

Validation par simulation MATLAB : PID, identification et commande MVC

Introduction

Ce chapitre présente la mise en œuvre pratique des concepts théoriques expliqués dans le chapitre précédent. L'objectif est d'étudier et de comparer les performances de deux stratégies de commande : la commande PID classique et la commande par réseau de neurones, appliquées à des systèmes de natures fondamentalement différentes.

La commande PID, dont la loi de commande repose sur la connaissance implicite du modèle du système, est dans un premier temps appliquée à des systèmes **linéaires invariants** (SLI). Le cas classique étudié ici est le système du second ordre de fonction de transfert :

$$H(s) \;=\; \frac{K\,\omega_n^2}{s^2 + 2\,\xi\,\omega_n\,s + \omega_n^2} \tag{2.0.1}$$

où K est le gain statique, ξ le coefficient d'amortissement et ω_n la pulsation naturelle. Pour ces systèmes, le régulateur PID permet d'assurer un bon suivi de consigne sous réserve d'un réglage approprié des paramètres (K_p, K_i, K_d).

Cependant, le régulateur PID atteint ses limites face aux systèmes **non linéaires**, pour lesquels aucune fonction de transfert ne peut représenter fidèlement le comportement. C'est pourquoi, dans un second temps, nous recourons à la commande neuronale par MLP, capable d'apprendre implicitement la dynamique du système à partir de données d'entrée-sortie.

Le système considéré est un **système non linéaire à mémoire**, dont la dynamique est la suivante :

$$y(k) \;=\; -\,\frac{0{,}8\;a(k)\;\Big[u(k-1)\Big]^2}{1+\Big[u(k-1)\Big]^2} + 0{,}033\;y(k-1) + 0{,}5\;u(k-1) \tag{2.0.2}$$

avec :

$$a(k) \;=\; y(k-2)$$

La non-linéarité est portée par le terme $\dfrac{a(k)\,[u(k-1)]^2}{1+[u(k-1)]^2}$, dont la saturation en entrée et la dépendance aux retards $y(k-1)$, $y(k-2)$ et $u(k-1)$ rendent toute linéarisation locale inadaptée.

L'ensemble de ces simulations est réalisé sous MATLAB et permet d'évaluer, pour chaque stratégie de

commande, les critères de performance obtenus.

2.1 Simulation et analyse de la commande PID numérique

Cette section présente l'étude comparative des performances du correcteur PID appliqué à un modèle de second ordre, représentatif de la dynamique de systèmes physiques réels.

2.1.1 Commande PID d'un système du second ordre

2.1.1.1 Présentation du système et discrétisation

On considère un système du second ordre de fonction de transfert :

$$H(s) \;=\; \frac{K_s\,\omega_0^2}{s^2 + 2\,\xi\,\omega_0\,s + \omega_0^2} \tag{2.1.1}$$

avec les paramètres numériques suivants :

TABLE 2.1 – Paramètres du système du second ordre.

Paramètre	Gain K_s	Pulsation propre ω_0	Amortissement ξ
Valeur	5	20 rad/s	0,5

Le coefficient d'amortissement $\xi = 0{,}5 < 1$ caractérise donc un régime **sous-amorti** du système : la réponse indicielle en boucle ouverte présente un dépassement avant d'atteindre le régime permanent, comme l'illustre la figure 2.1.

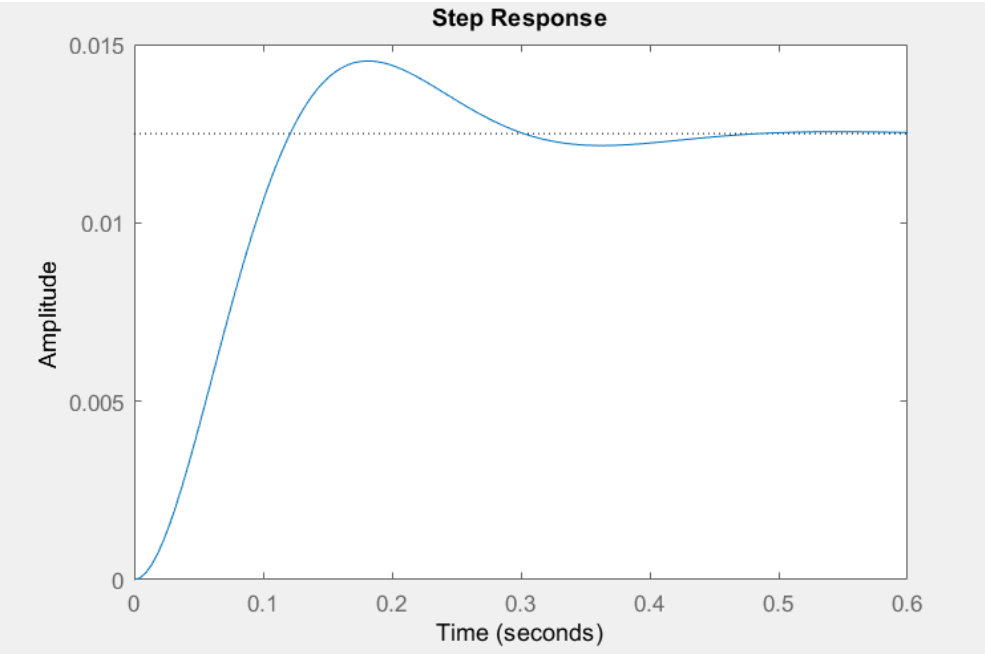


FIGURE 2.1 – Réponse indicielle du système du second ordre en boucle ouverte.

La période d'échantillonnage est choisie égale à $T_e = 0{,}36\,\text{s}$. La discrétisation par ZOH donne :

$$H(z) \;=\; \frac{b_1\, z^{-1} + b_2\, z^{-2}}{1 + a_1\, z^{-1} + a_2\, z^{-2}} \tag{2.1.2}$$

dont l'équation récurrente est :

$$y(k) \;=\; -a_1\, y(k-1) - a_2\, y(k-2) + b_1\, u(k-1) + b_2\, u(k-2) \tag{2.1.3}$$

2.1.1.2 Loi de commande PID discrète et calcul des coefficients

La même loi incrémentale est appliquée. Les coefficients retenus sont $c_0 = 15$, $c_1 = 1$, $c_2 = 1$. En inversant les relations :

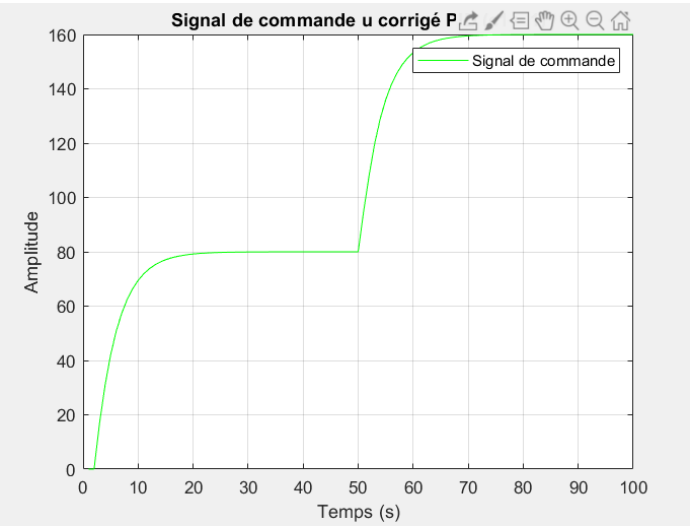
$$K_d \;=\; c_2 \cdot T_e \;=\; 0{,}36, \qquad K_p \;=\; -c_1 - \frac{2\,K_d}{T_e} \;=\; -3, \qquad K_i \;=\; \frac{c_0 - K_p - K_d/T_e}{T_e} \;\approx\; 47{,}2$$

TABLE 2.2 – Paramètres du correcteur PID discret pour le système du second ordre.

Coefficient discret	c_0	c_1	c_2
Valeur	15	1	1
Paramètre PID	$K_p = -3$	$K_i \approx 47{,}2$	$K_d = 0{,}36$

2.1.1.3 Résultats de simulation et interprétation

La simulation est conduite sur $N = 100$ échantillons avec le même signal de référence en échelon double que précédemment.



(a) Signal de commande $u(k)$ généré par le correcteur PID.



(b) Réponse $y(k)$ du système du second ordre régulé par PID.

FIGURE 2.2 – Simulation de la commande PID appliquée au système du second ordre ($K_s = 5$, $\omega_0 = 20\,\text{rad/s}$, $\xi = 0{,}5$).

L'évolution du signal de commande $u(k)$

- La commande évolue de façon exponentielle croissante jusqu'à atteindre la valeur finale dans le premier régime et dans le deuxième régime imposé par la consigne.

Réponse du système $y(k)$

- Le système adopte bien la réponse d'un système du premier ordre, grâce à l'action du correcteur PID. Sa réponse suit bien la consigne sans erreur statique au régime permanent, aussi bien dans le premier que dans le deuxième régime imposé.

Bilan de la commande PID Les simulations confirment l'efficacité du régulateur PID discret pour les systèmes linéaires invariants. Cependant, le réglage manuel des coefficients (c_0, c_1, c_2) suppose une connaissance implicite de la dynamique du système. Face à des systèmes non linéaires ou à modèle inconnu, cette approche devient inadaptée, ce qui motive le recours à la commande neuronale présentée dans la section suivante.

2.2 Identification par réseaux de neurones

2.2.1 Objectif et démarche

L'objectif de cette section est d'évaluer la capacité d'un réseau de neurones à **identifier** la dynamique de systèmes non linéaires à partir de leurs seules données d'entrée-sortie, sans connaissance préalable du modèle mathématique. Dans les deux cas, l'apprentissage est réalisé par l'**algorithme de descente de gradient par lots** (*batch gradient descent*), dont la convergence est évaluée à travers le critère quadratique :

$$C_r(p) \;=\; \sum_{k=3}^{N} \Big(g(k) - y(k)\Big)^2$$

où p désigne le numéro de l'époque (*epoch*), c'est-à-dire le numéro du passage complet sur l'ensemble des données.

Remarque : la sommation sur le critère $C_r(p)$ commence à $k = 3$ car la sortie $y(k)$ du système dépend des échantillons aux instants $k-1$ et $k-2$. Le script Matlab commence par $k \geq 1$, on commence donc à $k = 3$ pour éviter les indices négatifs.

2.2.2 Identification du système non linéaire à mémoire

2.2.2.1 Propriétés du système

Le signal $y(k)$ est généré à partir d'une commande aléatoire $u(k) \in [-0{,}5,\ 0{,}5]$ selon l'équation récurrente :

$$y(k) \;=\; -\frac{0{,}8\, y(k-2)\,[u(k-1)]^2}{1 + [u(k-1)]^2} + 0{,}033\, y(k-1) + 0{,}5\, u(k-1) \tag{2.2.1}$$

Ce système présente deux niveaux de complexité : une **non-linéarité statique** (terme quadratique saturant) et une **dynamique à mémoire** (dépendance aux retards $k-1$ et $k-2$).

2.2.2.2 Topologie du neurone d'identification

Pour modéliser cette dynamique, le neurone utilise un vecteur d'entrée composé de la commande passée et du retour de sortie. La fonction d'activation choisie est la tangente hyperbolique (tanh).

- **Vecteur d'entrée :** $\mathbf{X}(k) = [-1,\ u(k-1),\ y(k-1)]^\top$
- **Vecteur des poids :** $\mathbf{W} = [w_0,\ w_1,\ w_2]^\top$
- **Sortie prédite :** $\hat{y}(k) = \tanh\Big(-w_0 + w_1\, u(k-1) + w_2\, y(k-1)\Big)$

Choix du pas d'adaptation μ. Le pas μ est l'hyperparamètre central de la descente de gradient. Sa valeur conditionne directement la vitesse et la stabilité de la convergence. La condition théorique de convergence impose :

$$0 \ < \ \mu \ < \ \frac{2}{\lambda_{\max}}$$

où $\lambda_{\max}$ est la plus grande valeur propre de la matrice de corrélation des entrées. En pratique, trois régimes sont observés :

TABLE 2.3 – Effet du pas d'adaptation μ sur la convergence.

Valeur de μ	**Comportement**	**Risque**
$\mu = 0{,}001$	Convergence très lente	Arrêt prématuré
$\mu = 0{,}01$	Convergence régulière	Compromis optimal
$\mu = 0{,}1$	Convergence rapide	Oscillations possibles

2.2.2.3 Résultats et interprétation

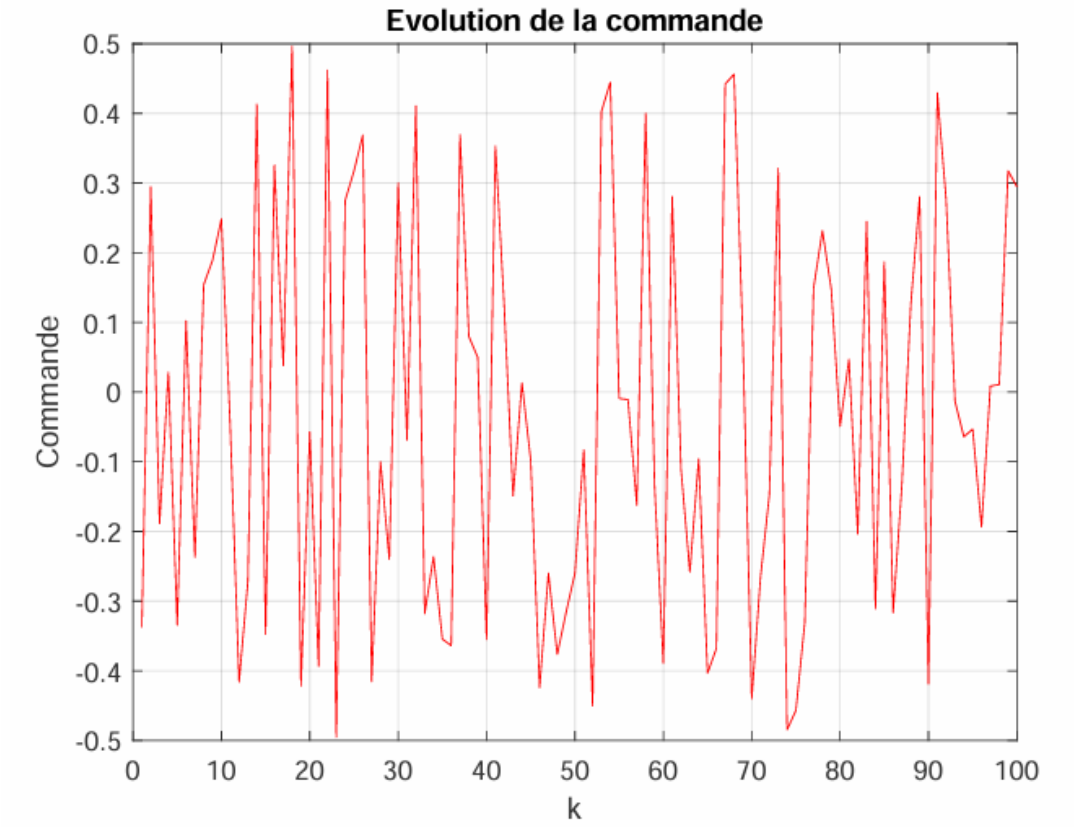
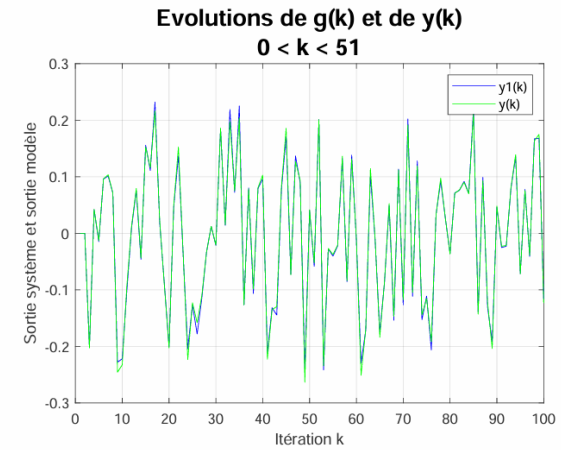
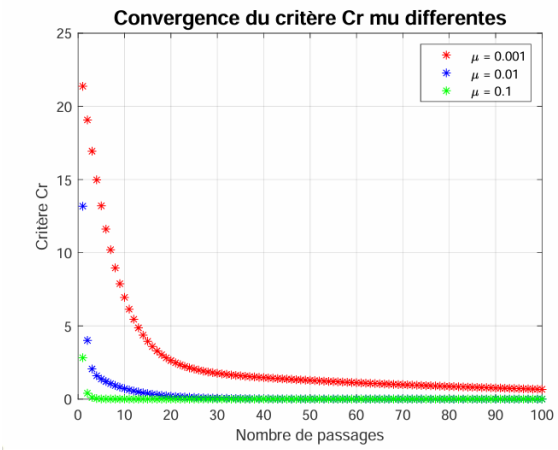


FIGURE 2.3 – Évolution de la commande aléatoire $u(k) \in [-0{,}5,\ 0{,}5]$.



(a) Évolution comparée de $y(k)$ et $\hat{y}(k)$.



(b) Évolution du critère C_r pour $\mu \in \{0{,}001,\ 0{,}01,\ 0{,}1\}$.

FIGURE 2.4 – Résultats de l'identification du système à mémoire.

Analyse de la convergence

- $\mu = 0{,}001$: La convergence du critère est extrêmement lente.
- $\mu = 0{,}01$: Décroissance régulière du critère C_r.

- $\mu = 0{,}1$: La convergence est la plus rapide.

Analyse du suivi Malgré la complexité de l'équation récurrente (2.2.1), le neurone parvient à suivre la dynamique du système. Les légers écarts observés s'expliquent par le fait que le modèle ne dispose que du retard $y(k-1)$ en entrée, alors que le système original dépend également de $y(k-2)$. La capacité de généralisation de la fonction tanh permet néanmoins de compenser partiellement cette information manquante.

TABLE 2.4 – Bilan comparatif de l'identification neuronale.

Critère	**Signal** $g(k)$	**Système à mémoire**
Type de non-linéarité	Statique multi-fréq.	Dynamique à retards
Entrées du neurone	$g(k-1)$, $g(k-2)$	$u(k-1)$, $y(k-1)$
Convergence	$p_{\max} \geq 50$	$\mu = 0{,}01$ optimal
Précision finale	$C_r \approx 0$	Suivi satisfaisant

2.3 Commande par minimum de variance

Après avoir identifié le modèle neuronal du système, nous mettons en œuvre une stratégie de commande par minimum de variance (MV). L'objectif est de trouver la commande $u(k)$ permettant à la sortie $y(k)$ du système d'atteindre la consigne $y_c(k)$ en minimisant les effets du bruit $e(k)$. On cherche à déterminer, à chaque instant k, la commande $u(k)$ optimale capable de minimiser l'écart entre la sortie prédite et une consigne imposée $y_c(k)$.

2.3.1 Critère de performance

Pour éviter des variations trop brusques de la commande qui pourraient déstabiliser le système réel, nous utilisons un critère quadratique incluant une pénalisation sur l'effort de commande :

$$J(u(k)) = [y_c(k+1) - \hat{y}(k+1)]^2 + \lambda[u(k) - u(k-1)]^2 \tag{2.3.1}$$

Où :

- $y_c(k+1)$ est la consigne à l'instant suivant.
- $\hat{y}(k+1)$ est la prédiction du modèle neuronal.
- λ est le facteur de pondération.

2.3.2 Algorithme d'optimisation itérative

Le modèle étant non linéaire (présence de la fonction tanh), la commande optimale $u(k)$ ne peut être extraite analytiquement. Nous utilisons donc un algorithme de descente de gradient interne à chaque

période d'échantillonnage, sur un nombre d'itérations $i_{max} = 10$:

Correction d'erreur : Afin d'améliorer la précision du prédicteur, nous introduisons un terme correctif α (facteur d'apprentissage ou de recalage) qui prend en compte l'erreur résiduelle entre la sortie réelle $y(k)$ et la prédiction précédente $\hat{y}(k)$:

$$\hat{y}(k+1) = \tanh(v(k)) + \alpha \cdot (y(k) - \hat{y}(k)) \tag{2.3.2}$$

Mise à jour de la commande : Le gradient du critère par rapport à la commande x (représentant $u(k)$) est calculé par :

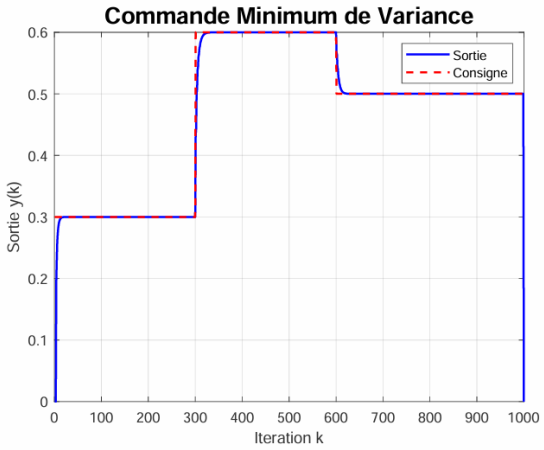
$$\frac{\partial J}{\partial x} = -2 \cdot (y_c(k+1) - \hat{y}(k+1)) \cdot \frac{\partial \hat{y}(k+1)}{\partial x} + 2\lambda(x - u(k-1)) \tag{2.3.3}$$

Avec $\frac{\partial \hat{y}(k+1)}{\partial x} = (1 - \tanh(v)^2) \cdot w_1$. La nouvelle valeur de la commande est ensuite mise à jour avec un pas $\mu = 0{,}2$:

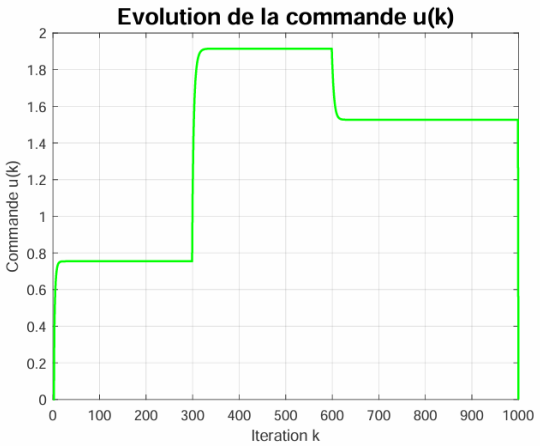
$$u(k)_{new} = u(k)_{old} - \mu \frac{\partial J}{\partial x} \tag{2.3.4}$$

2.3.3 Résultats et analyse du suivi

Le test a été effectué sur un profil de consigne en échelons (0,3, 0,6 et 0,5).



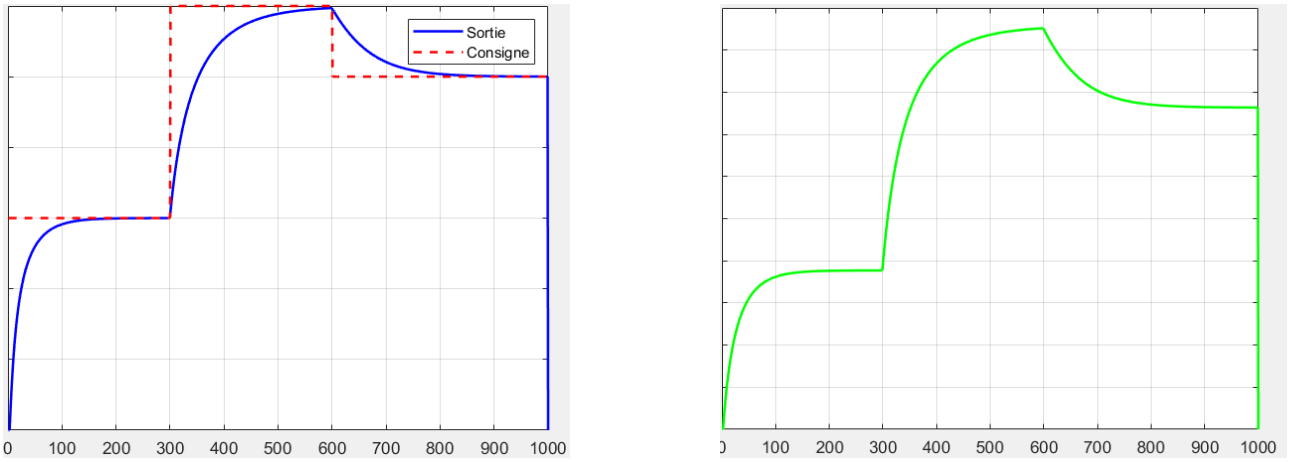
(a) Suivi de consigne par commande Minimum de Variance.



(b) Commande par minimum de variance.

FIGURE 2.5 – Résultats de la commande par minimum de variance.

Interprétation : Comme illustré sur la Figure 2.5a, la sortie $y(k)$ rejoint rapidement la consigne $y_c(k)$ malgré la forte non-linéarité du système réel. L'évolution de la commande $u(k)$ montre des pics d'accélération sans dépassement, suivis d'une stabilisation rapide vers le régime permanent. La valeur de λ très faible permet une réactivité élevée, tandis que l'algorithme itératif assure que le modèle neuronal est "inversé" correctement à chaque pas, garantissant ainsi un minimum de variance de l'erreur de suivi.



(a) Suivi de consigne pour les grandes valeurs de λ ($\lambda = 4$).

(b) Évolution de la commande par minimum de variance pour $\lambda = 4$.

FIGURE 2.6 – Changement de la réactivité du système pour les grandes valeurs de régularisation sur la commande.

Dans la figure 2.6, on observe qu'une forte régularisation sur la commande a influencé la rapidité du système et sa précision. En effet, d'après la figure 2.6b, la commande n'a pas encore atteint le régime permanent pendant le premier et le deuxième changement de consigne, et la réponse du système est en grand retard par rapport à la consigne.

Conclusion

Ce chapitre a présenté une étude comparative des deux stratégies de commande étudiées dans ce travail. La commande PID discrète, appliquée au système du second ordre, a démontré ses bonnes performances sur les systèmes linéaires invariants : erreur statique nulle, absence de dépassement et suivi correct de consigne variable. La principale limite réside dans la nécessité d'un réglage manuel des paramètres, qui suppose une connaissance de la dynamique du système.

Face aux systèmes non linéaires, la commande neuronale par descente de gradient a montré sa capacité à identifier implicitement la dynamique à partir des seules données d'entrée-sortie. Les résultats obtenus valident l'efficacité de l'algorithme de rétropropagation.

Ces résultats de simulation constituent le fondement théorique nécessaire à l'implémentation expérimentale présentée dans le chapitre suivant, qui portera sur l'application de ces méthodes à la régulation thermique réelle de l'enceinte.

Chapitre 3

Implémentation pratique

3.1 Introduction

Dans ce chapitre, nous présentons la réalisation pratique de notre projet. Après avoir étudié et simulé les différentes méthodes de commande dans l'environnement MATLAB, nous passons à l'implémentation sur le système réel. Cette étape consiste à mettre en œuvre la commande neuronale sur une carte ESP32 et à établir la communication avec une deuxième carte chargée de l'acquisition des données du système thermique à l'aide du capteur DHT22. Nous décrivons aussi les différents composants utilisés, l'architecture du système ainsi que les résultats expérimentaux obtenus.

3.2 Architecture matérielle

3.2.1 Carte de développement ESP32

L'ESP32 est un microcontrôleur puissant et peu coûteux, bien adapté à notre projet, fabriqué par la société Espressif Systems. Nous avons choisi cette carte car elle intègre un module Wi-Fi et Bluetooth. On a utilisé l'ESP32 pour piloter le système et assurer une connexion sans fil.

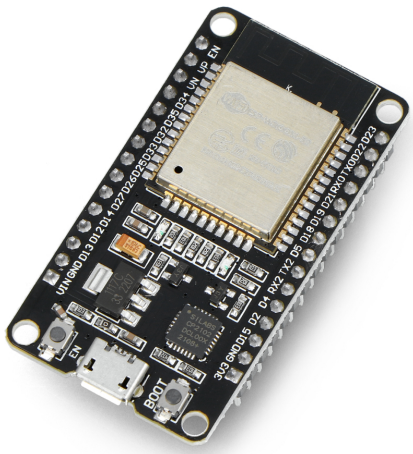


FIGURE 3.1 – Le microcontroleur ESP32

Ses principales caractéristiques sont présentées dans le tableau suivant :

Table 3.1 – Extraits du datasheet du microcontroleur ESP32

Caractéristique	**Description**
Microcontrôleur	ESP32 (double cœur)
Fréquence CPU	Jusqu'à 240 MHz
Mémoire RAM	520 KB SRAM
Mémoire Flash	4 MB
Connectivité	Wi-Fi 802.11 b/g/n, Bluetooth v4.2
Tension de fonctionnement	3.3 V
Entrées/Sorties GPIO	Jusqu'à 34 broches
ADC (Convertisseur analogique-numérique)	12 bits
DAC (Convertisseur numérique-analogique)	2 canaux
Interfaces de communication	UART, SPI, I2C, CAN
Consommation	Faible consommation (mode veille disponible)

3.2.2 AC Dimmer Module

Le module AC Dimmer (Gradateur) est un contrôleur de puissance pour charges alternatives (AC) 220 V, utilisant une technologie TRIAC à coupure de phase. Compatible avec Arduino/ESP32, il gère 1 canal (8 A) via une logique 3.3 V/5 V, détecte le passage par zéro pour une variation précise (lumière, moteurs). On a utilisé ce module pour contrôler l'intensité d'une lampe.

Les pins du module AC Dimmer :

- **VCC** et **GND** : pour l'alimentation du module.
- **Z-C** : permet de détecter le passage par zéro de la tension du secteur. Elle génère un signal numérique à chaque fois que la tension alternative atteint 0 V, ce qui permet au microcontrôleur de se synchroniser avec le réseau électrique. Cette synchronisation est essentielle pour assurer une commande précise du TRIAC et réduire les perturbations électriques.
- **PSM** : utilisée pour recevoir le signal de commande (MLI) provenant de l'ESP32. Ce signal permet de contrôler la puissance fournie à la charge en ajustant le moment de déclenchement du TRIAC après le passage par zéro.



FIGURE 3.2 – Module actionneur à Triacc (Dimmer)

3.2.3 Capteur de température DHT22

Le DHT22 est un capteur numérique utilisé pour mesurer la température et l'humidité de l'air. Dans notre projet, il est utilisé pour mesurer la température du système thermique et envoyer les données à la carte ESP32 pour le traitement et la commande.

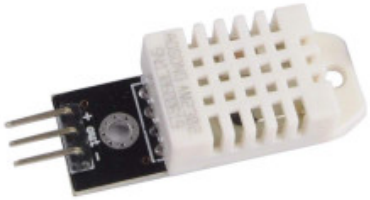


FIGURE 3.3 – Capteur d'humidité et de température DHT22

TABLE 3.2 – Caractéristiques du capteur DHT22

Caractéristique	**Valeur / Description**
Type de capteur	Température et humidité
Plage de température	−40°C à +80°C
Précision température	±0.1°C
Plage d'humidité	0% à 100% RH
Précision humidité	±2 à 5% RH
Tension d'alimentation	3.3 V à 5 V

3.2.4 Equipement de chauffage 220V

On utulise comme effecteur de chauffage pour le systéme une lampe 220 V, pour varier la température dans une boîte fermée. Elle est efficace car elle reçoit une puissance importante du module Dimmer.

3.3 Architecture du système

L'architecture générale du système thermique est représentée par la figure suivante :

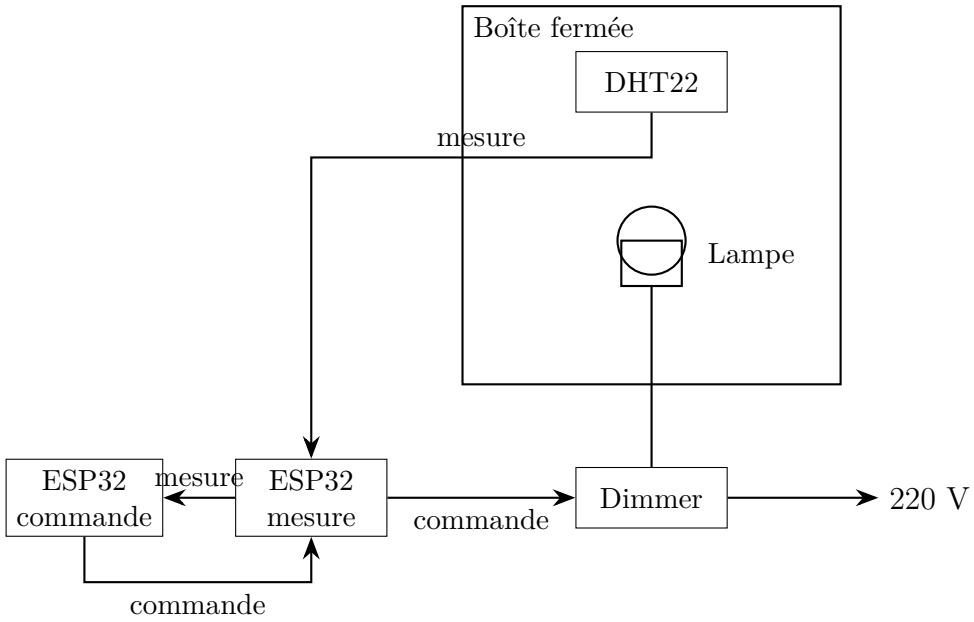


FIGURE 3.4 – Schéma synoptique du système de régulation thermique

Le système est composé de deux cartes ESP32 qui communiquent entre elles sans fil. La première carte est utilisée pour mesurer la température du système thermique à l'aide du capteur DHT22 et envoyer ces données à la deuxième carte. La deuxième carte reçoit les informations, exécute l'algorithme de commande basé sur les réseaux de neurones et calcule l'action de commande. Cette action est ensuite envoyée vers un module dimmer AC qui contrôle la puissance appliquée à une lampe 220 V, permettant ainsi de réguler la température du système. Cette architecture permet de séparer l'acquisition des données et le traitement de la commande pour assurer que l'utilisateur contrôle le système à distance.

3.3.1 Essai en boucle ouverte

Tout d'abord, dans le cadre de notre travail pratique, nous avons commencé par réaliser un essai en boucle ouverte sur le système thermique, où une commande constante notée u a été appliquée sur des intervalles de 2000 échantillons, avec un échantillonnage toutes les 2 secondes grâce au capteur DHT22. Les températures mesurées ont été sauvegardées dans le but de créer une base d'apprentissage, qui nous sert ensuite à entraîner un réseau de neurones artificiels utilisé sur une carte ESP32, dont les poids sont indispensables à la mise en œuvre de notre algorithme de commande à minimum de variance. De cette

manière, nous avons non seulement appris à comprendre le comportement du système mais aussi préparé la phase de commande intelligente.

Les données enregistrées après l'essai en boucle ouverte sont présentées dans la figure suivante.

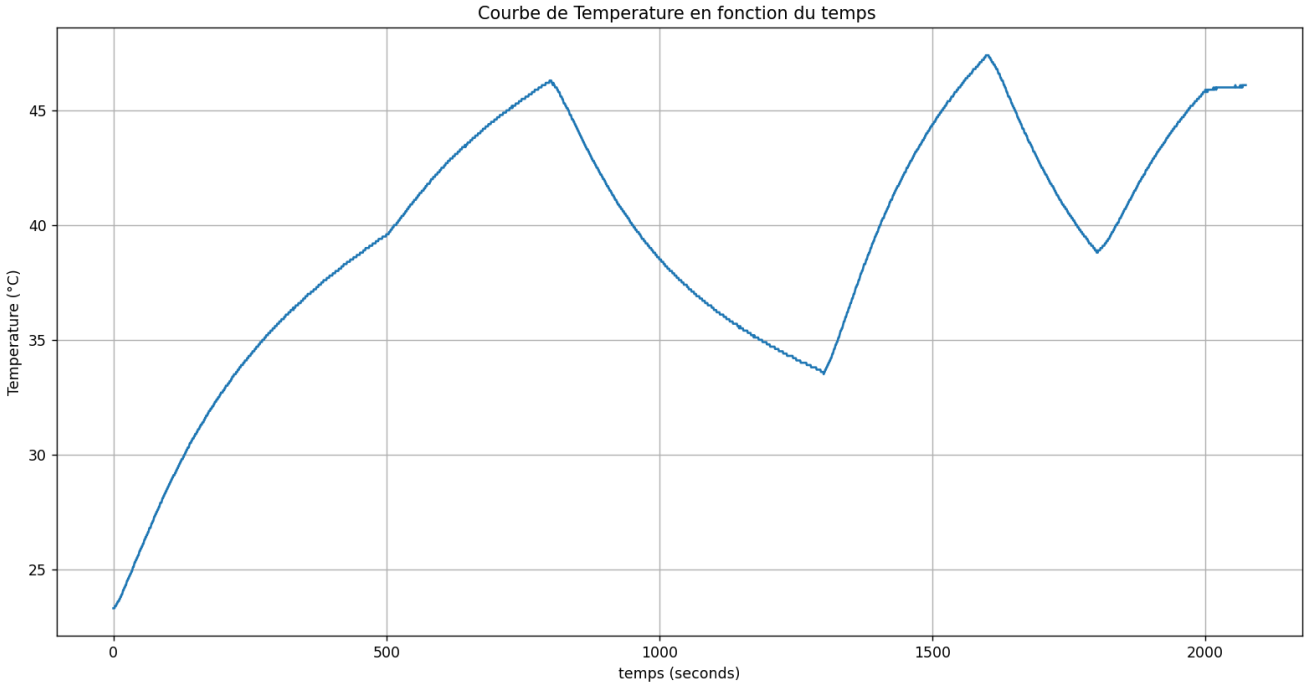


FIGURE 3.5 – Evolution de la température en fonction du temps

3.4 Apprentissage du neurone artificiel

Dans cette partie, nous avons utilisé les données obtenues lors de l'essai en boucle ouverte pour entraîner un neurone artificiel. Ces données, qui représentent l'évolution de la température en fonction de la commande appliquée, ont été traitées sous MATLAB pour constituer un jeu d'apprentissage. Ensuite, un algorithme d'apprentissage a été mis en place pour ajuster les poids du neurone afin de minimiser l'erreur entre la sortie réelle et la sortie estimée. Les résultats obtenus sont présentés sous forme de figures dans les sections suivantes, afin d'analyser la qualité de l'apprentissage et la capacité du modèle à reproduire le comportement du système thermique.

Tout d'abord, nous avons ouvert les fichiers qui contiennent les données enregistrées et avons parcouru ces valeurs pour les stocker dans un vecteur. Les valeurs de température ont été divisées par 100 pour les normaliser entre 0 et 1. Cela facilite le processus d'apprentissage et améliore la stabilité de l'algorithme.

Ensuite, nous avons initialisé les variables nécessaires à l'apprentissage, comme les poids du neurone et les paramètres de l'algorithme. Nous avons également introduit les valeurs de la commande u, utilisées lors de l'essai en boucle ouverte, et les avons normalisées en les divisant par 5. Cela nous permet d'avoir des données d'entrée cohérentes avec celles de sortie.

Après cela, nous avons mis en œuvre l'algorithme principal d'apprentissage. Cet algorithme ajuste les poids du neurone de manière itérative pour minimiser l'erreur entre la sortie réelle du système et la

sortie estimée par le modèle.

Une fois l'apprentissage réalisé, une phase de test qui suit permet de valider encore l'efficacité du modèle choisi.

Enfin, les résultats de simulation sont affichés sous forme de courbes. Cela nous permet de visualiser la qualité de l'apprentissage et la précision du modèle. Ces figures montrent la capacité du neurone à reproduire le comportement du système thermique.

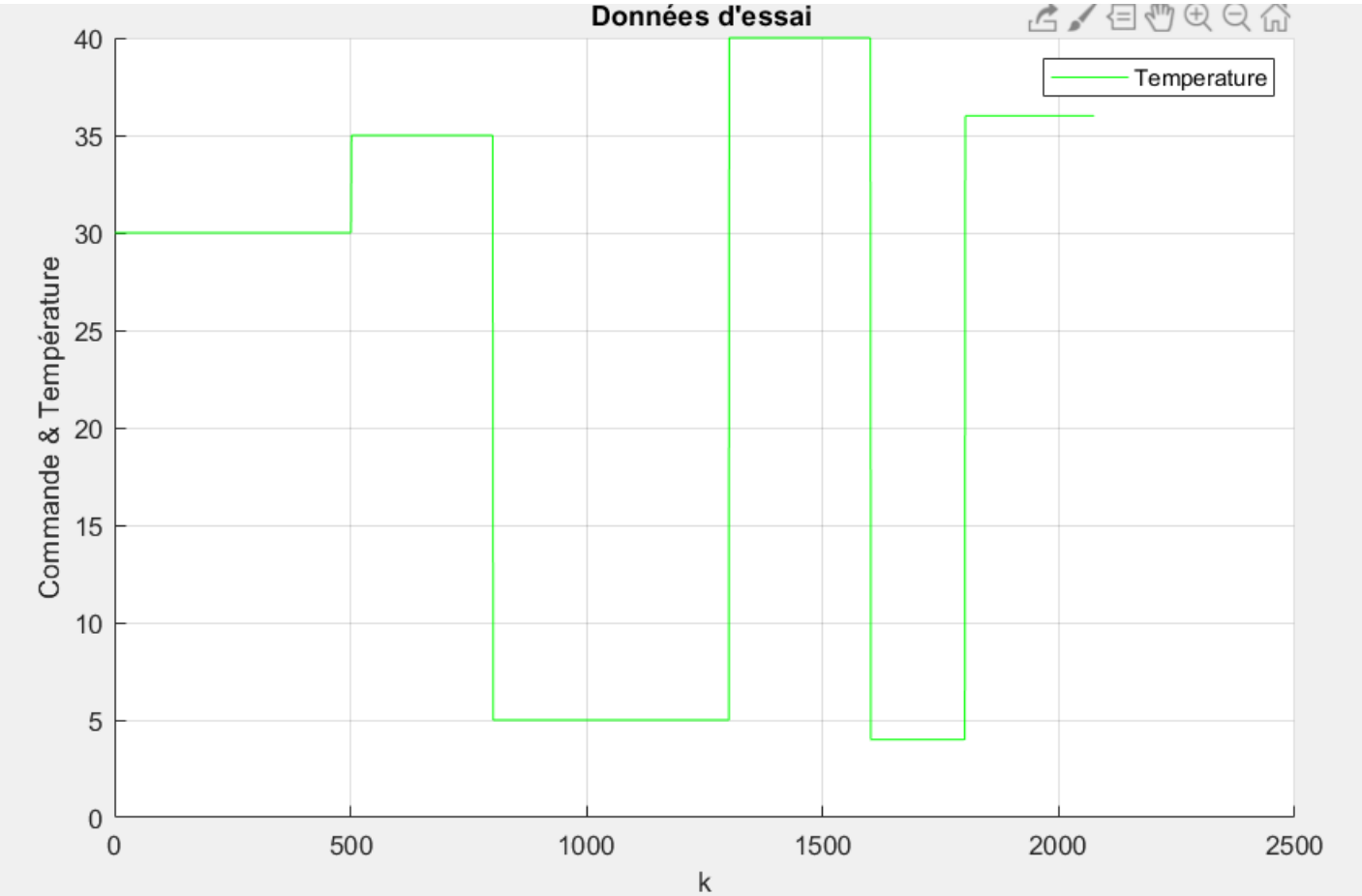


FIGURE 3.6 – Évolution de la commande en puissance pendant l'essai en boucle ouverte

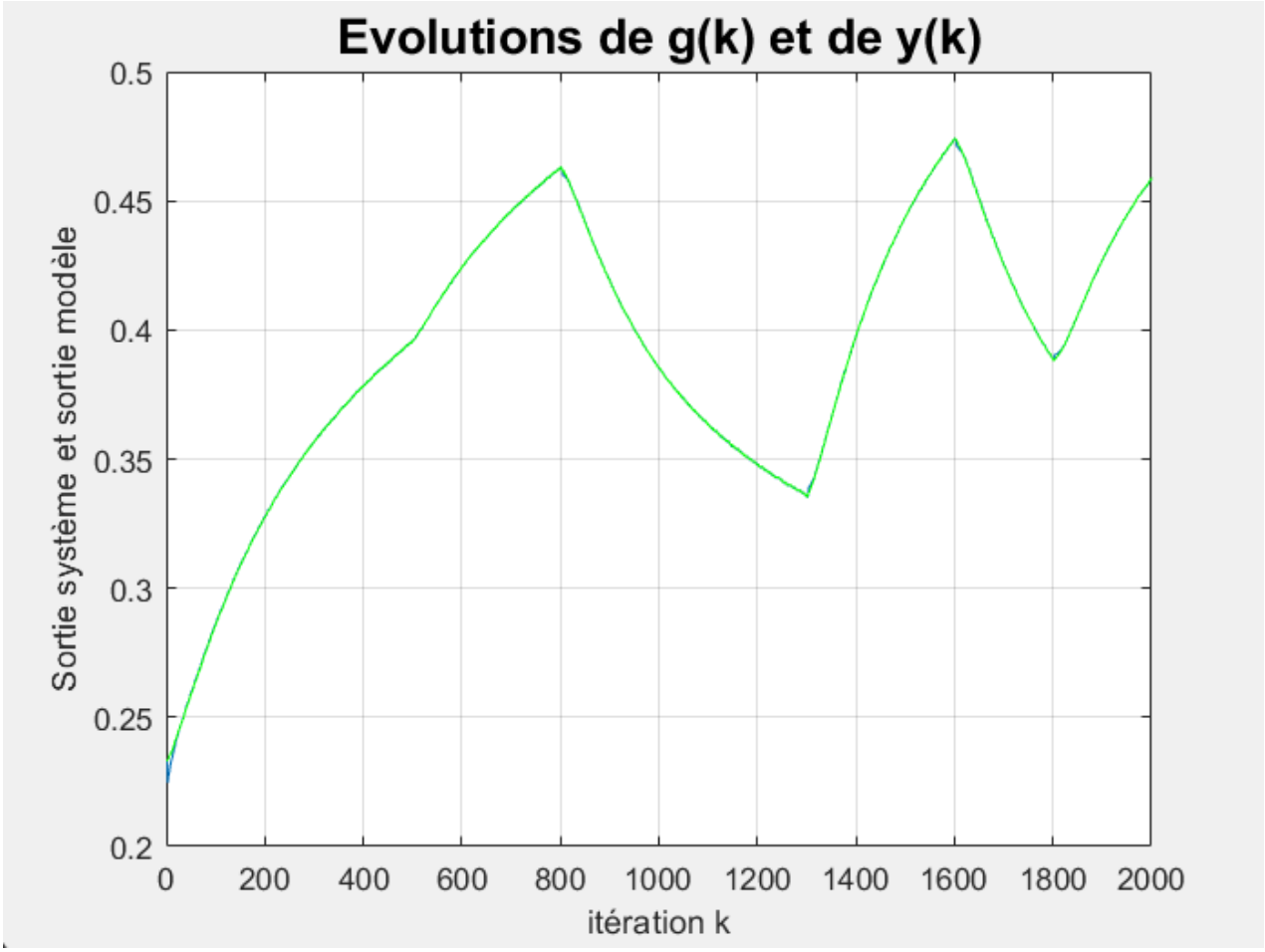


FIGURE 3.7 – Évolution de la sortie système et de la sortie prédite du neurone pendant l'apprentissage

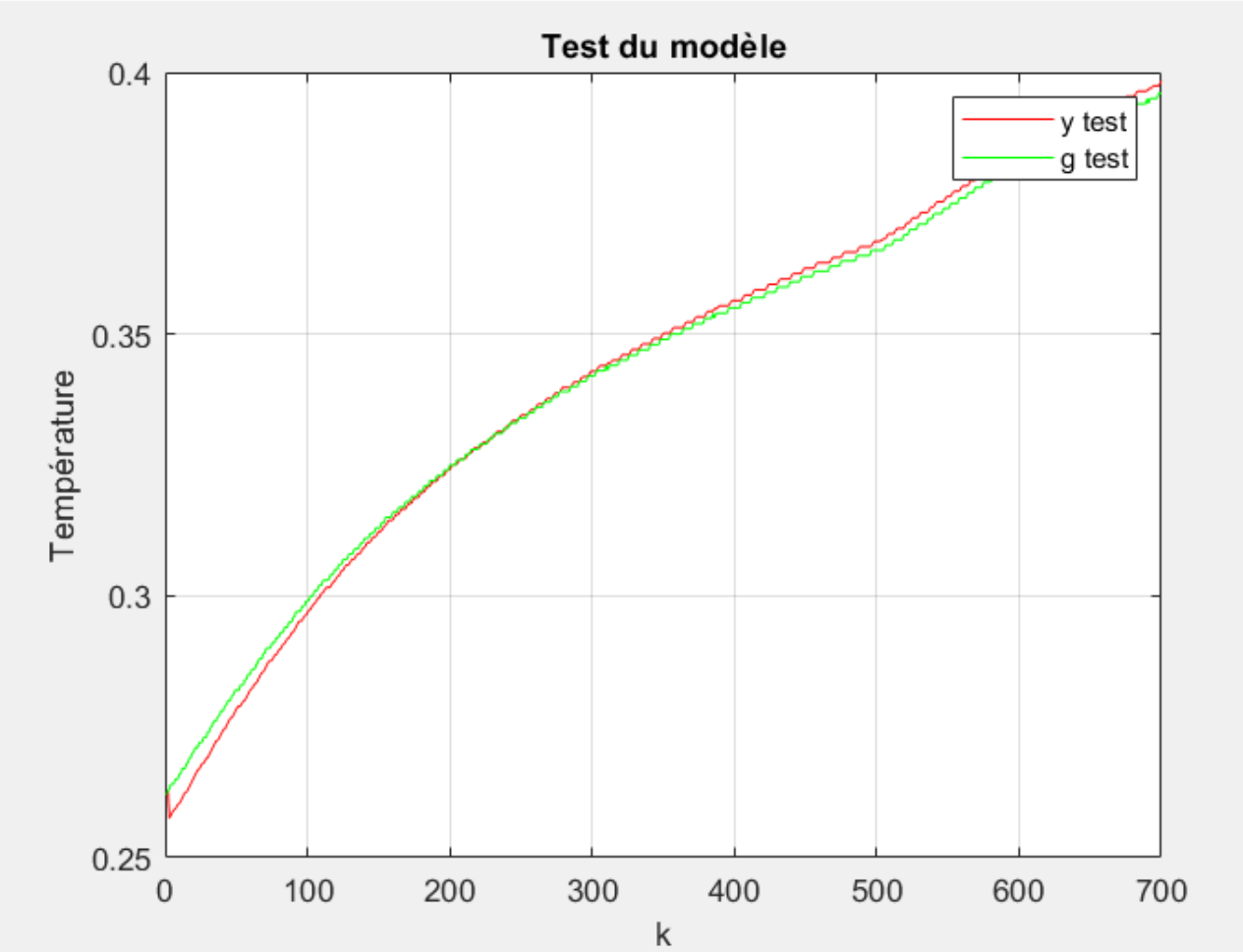


FIGURE 3.8 – Évolution de la sortie système et de la sortie prédite du neurone pendant le test

3.5 Essai en boucle fermée

Après la phase d'apprentissage, nous passons à un essai en boucle fermée en utilisant une commande à minimum de variance. Cet algorithme de commande exploite les poids obtenus lors de l'apprentissage du neurone artificiel afin d'estimer le comportement du système et de calculer l'action de commande optimale. Ainsi, le système devient capable d'ajuster automatiquement la puissance appliquée pour suivre la consigne de température tout en minimisant l'erreur et les variations indésirables.

3.5.1 Implémentation de l'algorithme sur la carte

Dans cette étape, nous avons mis en œuvre l'algorithme de commande à minimum de variance sur la carte ESP32. Cet algorithme utilise les poids que nous avons obtenus lors de la phase d'apprentissage du neurone artificiel pour estimer le comportement du système et calculer la commande à appliquer.

- **RBDdimmer** : cette bibliothèque donne des fonctions prédéfinies pour contrôler le dimmer.
- **DHT** : permet de lire les valeurs de température mesurées par le capteur DHT22 avec la fonction prédéfinie `dht.readTemperature()`.

Le programme sur la carte ESP32 est conçu de manière simple pour fonctionner en temps réel. Tout d'abord, il initialise les variables et configure les entrées et les sorties ainsi que les communications nécessaires. Ensuite, le programme lit la température mesurée par le capteur DHT22. Il utilise les poids du neurone (w_0, w_1, w_2, w_3 et w_4) obtenus lors de la phase d'apprentissage pour calculer la commande. Cette commande est ensuite envoyée au module de puissance dimmer pour contrôler le système thermique. Une boucle infinie répète ces étapes en continu pour réguler la température.

3.5.2 Montage expérimental

La broche **data** du capteur DHT22 transmet la valeur de la température à la carte ESP32. Cette dernière calcule ensuite la commande de contrôle. Elle attend ensuite le signal de synchronisation provenant de la broche **Z-C** du dimmer, puis envoie la commande via la broche **PSM**. Le dimmer module alors la puissance issue du secteur et l'applique à la lampe, ce qui modifie la température dans la boîte isolée. Ce cycle se répète en continu afin d'assurer la régulation et de permettre à la température de suivre la consigne.

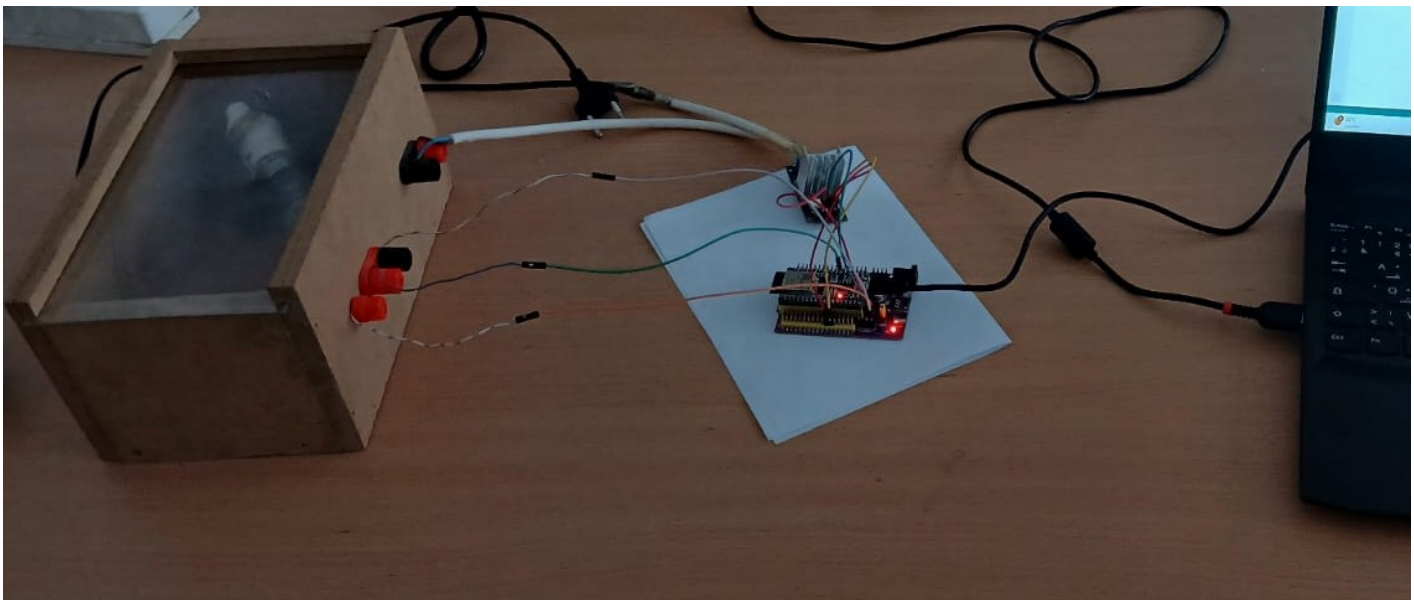


FIGURE 3.9 – Montage expérimental

3.5.3 Déroulement de l'essai

Après avoir mis en place le système, nous avons effectué un essai en boucle fermée. L'objectif principal de cet essai était de réguler la température en suivant une consigne donnée. Le système fonctionne en temps réel, avec une mesure de la température qui est prise toutes les 2 secondes. Pendant l'expérience, nous avons enregistré un total de 4000 échantillons afin d'analyser le comportement du système sur une durée suffisante.

Les valeurs mesurées, qui représentent les données de l'essai expérimental sur le système, telles que la température et le signal de commande, ont été récupérées à partir de la carte ESP32 et stockées dans un

fichier.

3.5.4 Résultats expérimentaux et analyse

Nous avons importé les valeurs enregistrées dans MATLAB pour réaliser des simulations. Des courbes ont été tracées pour représenter l'évolution de la température et la commande. La consigne donnée est 35°C puis 45°C.

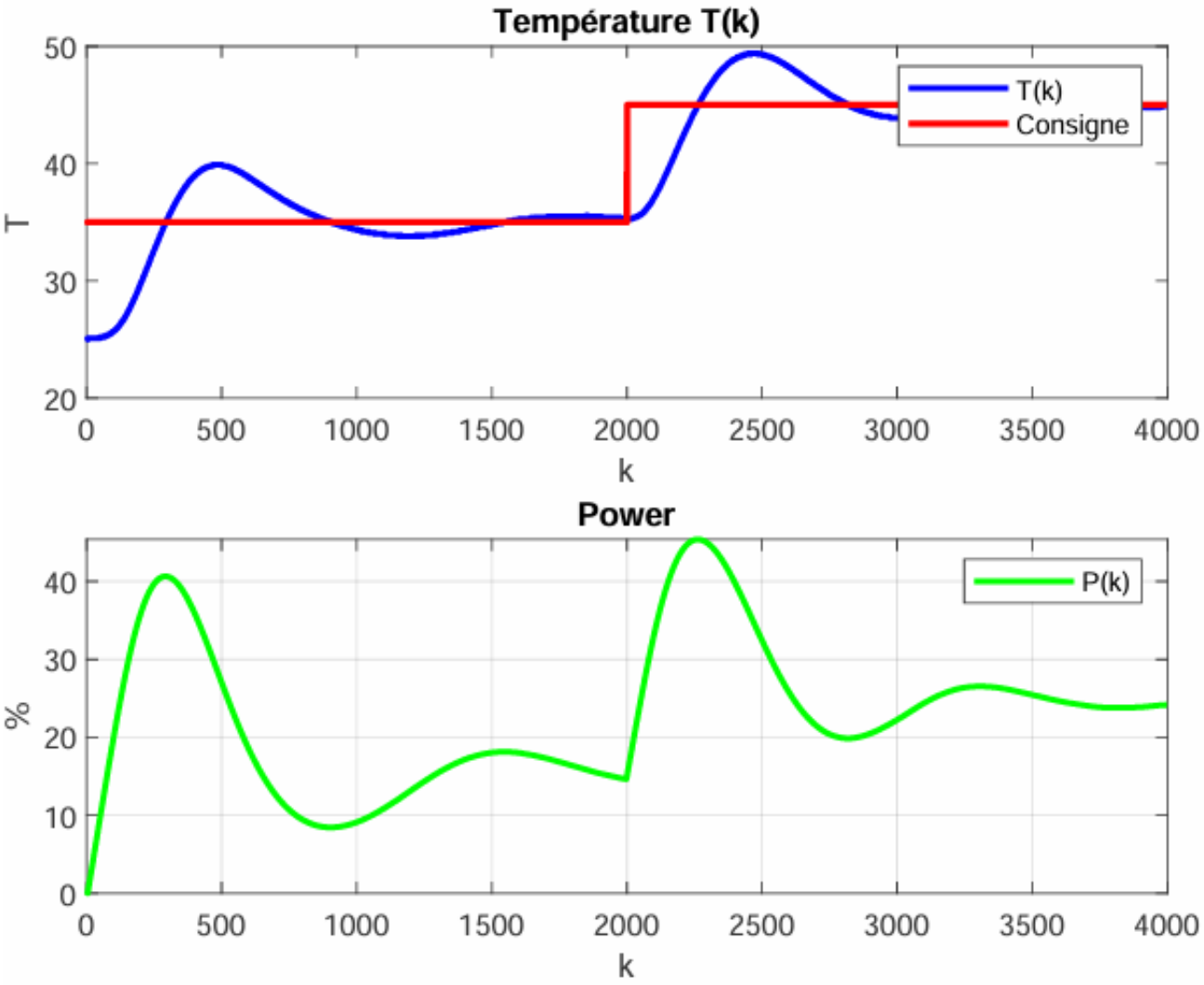


FIGURE 3.10 – Évolution de la sortie, de la consigne et de la commande pour l'essai en boucle fermée

Nous observons que la température finit par suivre correctement la consigne, ce qui montre que le système de régulation fonctionne. Cependant, cette stabilisation n'est pas immédiate, elle apparaît seulement après un certain temps de fonctionnement. De plus, on remarque la présence de petites oscillations autour de la consigne. Ces variations sont dues aux limitations du système réel, comme les délais de réponse, les perturbations externes et la nature même de la commande utilisée. Malgré ces oscillations, le système reste globalement stable et capable de maintenir la température proche de la

valeur souhaitée.

3.6 Communication sans fil

3.6.1 Le protocole ESP-NOW

Nous avons utilisé le protocole **ESP-NOW** pour assurer la communication sans fil.

ESP-NOW est un protocole de communication sans fil développé par Espressif pour les microcontrôleurs ESP32. Il permet aux appareils d'échanger les données directement. La communication se fait sur la bande de fréquence 2,4 GHz, avec une latence très faible, de l'ordre de la milliseconde. Cela le rend très utile pour notre système puisqu'il nécessite une réponse rapide.

TABLE 3.3 – Caractéristiques du protocole ESP-NOW

Caractéristique	**Valeur / Description**
Fréquence	2.4 GHz
Latence	Très faible ($\sim$ ms)
Portée	$\sim$200 m en espace ouvert
Taille max des données	250 octets par trame
Réseau Wi-Fi requis	Non
Consommation énergétique	Très faible

3.6.2 Implémentation des algorithmes de commandes sur les microcontroleurs ESP32

Dans cette étape finale, nous avons divisé l'algorithme de commande à minimum de variance sur deux cartes ESP32. L'une pour la mesure de la température et l'autre pour la commande de notre système thermique. Nous avons juste ajouté des parties dans le code pour assurer la communication entre les deux cartes à travers le protocole ESP-NOW, mais l'algorithme principal de commande à minimum de variance est le même.

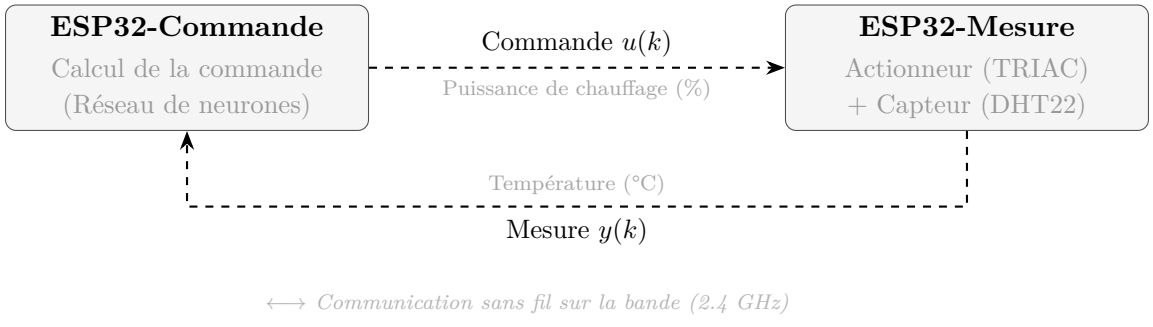


FIGURE 3.11 – Schéma de communication entre les deux ESP32

Ce schéma montre comment les deux microcontrôleurs ESP32 échangent les données avec le protocole ESP-NOW. La carte ESP32-commande sert de contrôleur. Il commence par identifier la carte ESP32-mesure grâce à son adresse MAC unique. Ensuite, elle initialise le protocole et enregistre les fonctions de rappel qui se déclenchent automatiquement à chaque envoi ou réception de message.

La carte ESP32-commande attend la réception de la température envoyée par ESP32-mesure. Elle utilise la valeur mesurée de la température pour calculer la commande optimale, puis envoie le signal de puissance en retour.

3.6.3 Montage expérimental

Dans ce deuxième montage, nous avons ajouté seulement l'autre carte ESP32, tout le reste est le même.

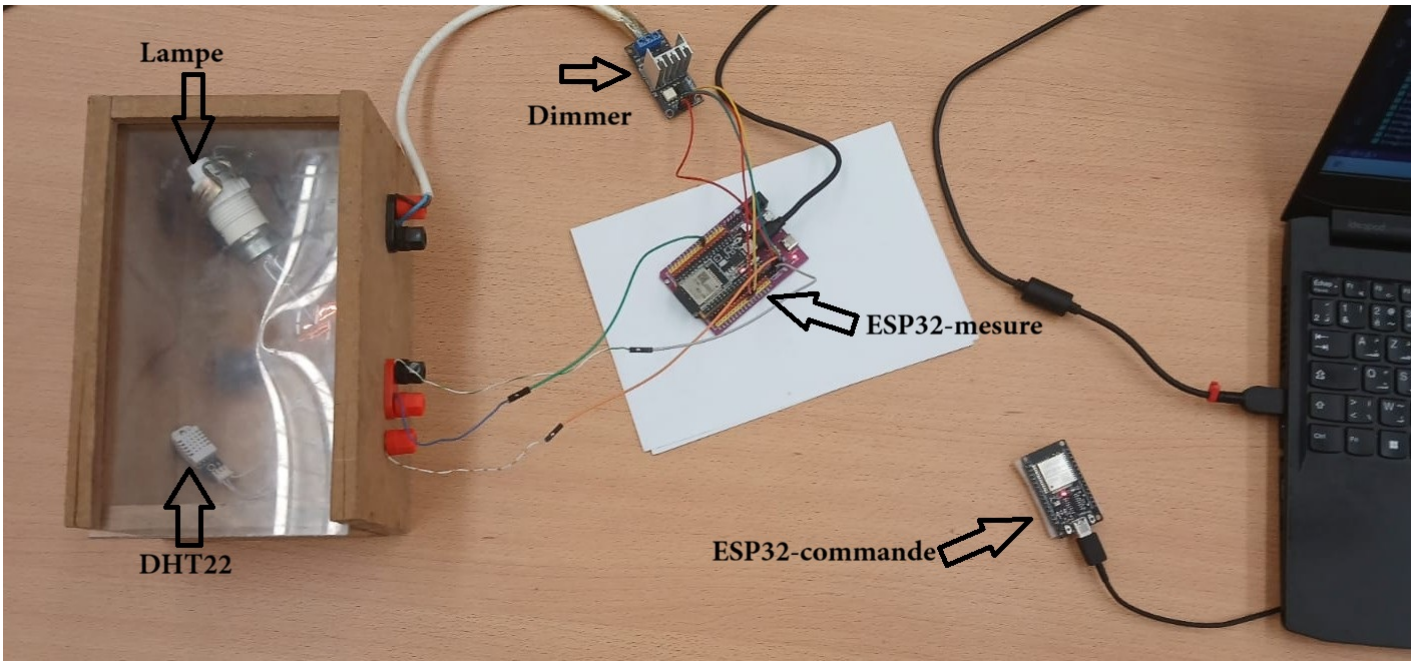


FIGURE 3.12 – Montage expérimental avec communication

3.6.4 Résultats expérimentaux et analyse

Après la mise en place du système, nous avons réalisé l'essai en boucle fermée en utilisant la communication sans fil entre les deux cartes ESP32. Au cours de cet essai, un total de 4000 mesures a été enregistré afin d'étudier le comportement du système et d'évaluer les performances de la commande dans un environnement avec communication sans fil.

Les mesures enregistrées sont importées dans MATLAB pour réaliser les simulations. La consigne donnée est 40°C et 50°C. Nous avons donc tracé les courbes suivantes.

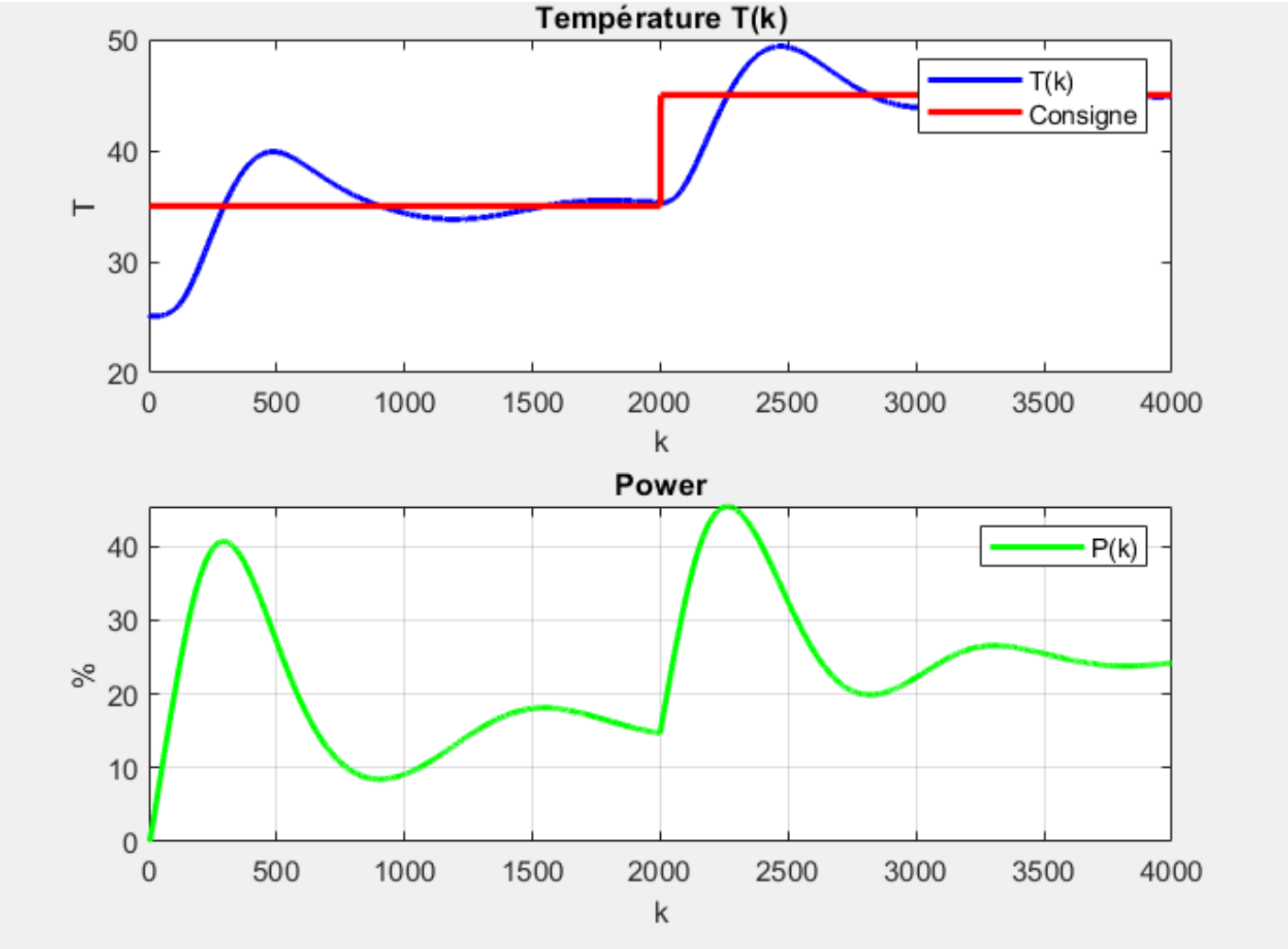


FIGURE 3.13 – Évolution de la sortie, de la consigne et de la commande

D'après les résultats obtenus, nous constatons que la température suit la consigne, mais avec un certain temps de retard. Cela signifie que le système a besoin d'un délai avant de réagir et d'atteindre la valeur souhaitée, ce qui est normal pour un système thermique à cause de son inertie. Une fois proche de la consigne, on observe la présence de petites oscillations autour de cette valeur. Ces variations sont liées aux limites du système réel et à la dynamique de la commande, mais elles restent faibles et n'affectent pas la stabilité globale. En effet, le système demeure stable et continue de fonctionner correctement.

Conclusion

Dans ce chapitre, nous avons expliqué comment nous avons réalisé l'essai pratique de la commande à minimum de variance en commençant par l'essai expérimental sur le système pour extraire sa dynamique, et faire l'apprentissage du modèle choisi avec ses données jusqu'à arriver à faire sa commande par minimum de variance à distance en utilisant deux microcontrôleurs ESP32.

D'une part, les résultats montrent la cohérence du modèle choisi et l'efficacité de la méthode de commande introduite qui permet au système de suivre la consigne ; d'autre part, ils montrent la réponse

lente des systèmes thermiques en général, ce qui ouvre des perspectives sur d'autres méthodes de commande plus efficaces pour ce type de système.

Conclusion générale

Ce projet de fin d'année avait pour objectif la conception et l'implémentation d'une stratégie de commande neuronale à minimum de variance appliquée à un système thermique réel.

Dans un premier temps, on a établit les fondements théoriques nécessaires pour cette approche. Nous avons introduit la commande PID appliquée à un systéme de deuxiéme ordre.Cependant,on a montré alors sa limite face à des systémes plus dynamique et de modèle connu.Face à des systèmes non linéaires, cette approche atteint ses limites, ce qui a motivé le recours aux réseaux de neurones artificiels. Dans un second temps,nous avons simulé la commande par réseaux de neurones sous MATLAB. L'algorithme de descente de gradient par lots a permis d'identifier avec précision la dynamique du systéme non linéaire, avec une convergence optimale du critére d'erreur.

Enfin, Nous avons réalisé l'essai pratique sur carte ESP32 commençant par une commande cablée sur le systéme appliquant en fait ce qu'on vue dans la démarche théorique du projet,par la suite nous avons établie une commande à communication sans fil sur le systéme.

Ce travail ouvre plusieurs perspectives d'amélioration. Le prototype étudié est un système thermique lentement variable, ce qui rend l'hypothèse d'une dynamique localement linéarisable acceptable sur cet intervalle de fonctionnement. De plus, l'excitation d'un neurone de type perceptron durant la phase d'apprentissage a produit des résultats satisfaisants : l'erreur finale lors de la phase de test est restée dans des limites lisibles et acceptables pour ce type de système. Toutefois, cette approche atteint ses limites dès lors que l'on s'intéresse à des systèmes à dynamique plus rapide ou fortement non linéaire. Dans ce cas, un perceptron à couche unique ne dispose pas d'une capacité de représentation suffisante pour capturer l'ensemble des variations du système : les informations acquises sur la dynamique entrée-sortie deviennent insuffisantes, et l'erreur d'identification devient non négligeable. Le recours à un réseau de neurones multicouche (MLP profond) s'impose alors, au prix d'un nombre de paramètres plus élevé et d'une complexité algorithmique accrue. Cette complexité soulève à son tour des contraintes d'implémentation embarquée : la charge de calcul augmente à chaque propagation avant et à chaque mise à jour des poids, et l'empreinte mémoire du modèle peut dépasser les capacités d'un microcontrôleur tel que l'ESP32. Une piste naturelle consiste alors à délocaliser la phase d'apprentissage sur un ordinateur hôte en exploitant des bibliothèques Python de machine learning optimisées — telles que TensorFlow Lite ou scikit-learn — puis à déployer uniquement le modèle entraîné sur la carte embarquée pour l'inférence en temps réel.

Bibliographie

[1] McCulloch, W. S., & Pitts, W. (1943). "A logical calculus of the ideas immanent in nervous activity". *Bulletin of mathematical biophysics*, 5(4), 115-133. https://link.springer.com/article/10.1007/BF02478259

[2] Rosenblatt, F. (1958). "The perceptron : a probabilistic model for information storage and organization in the brain". *Psychological review*, 65(6), 386. https://psycnet.apa.org/doi/10.1037/h0042519

[3] Minsky, M., & Papert, S. (1969). *Perceptrons : An Introduction to Computational Geometry*. MIT Press. https://archive.org/details/perceptronsintrod0000mins

[4] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). "Learning representations by back-propagating errors". *Nature*, 323(6088), 533-536. https://www.nature.com/articles/323533a0

[5] Cybenko, G. (1989). "Approximation by superpositions of a sigmoidal function". *Mathematics of Control, Signals and Systems*, 2(4), 303-314. https://link.springer.com/article/10.1007/BF02551274

[6] Narendra, K. S., & Parthasarathy, K. (1990). "Identification and control of dynamical systems using neural networks". *IEEE Transactions on neural networks*, 1(1), 4-27. https://ieeexplore.ieee.org/document/52541

[7] Åström, K. J., & Hägglund, T. (1995). *PID Controllers : Theory, Design, and Tuning*. Instrument Society of America. https://www.ucg.ac.me/skladiste/blog_2146/objava_92847/fajlovi/Astrom.pdf

[8] Mohan, N., Undeland, T. M., & Robbins, W. P. (2019). *Power Electronics : Converters, Applications, and Design*. 4th ed., Wiley.

[9] GitHub Pages. (2026). *Évaluation de la Dynamique des Systèmes*. https://usergeweb.github.io/015_dy_07_01_eval_dynamique.pdf

[10] Espressif Systems. (2025). *ESP32 Series Datasheet*. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[11] Aosong Electronics. (2018). *DHT11 Humidity and Temperature Sensor Datasheet*.

[12] MathWorks. *Deep Learning Toolbox User's Guide*. https://fr.mathworks.com/help/deeplearning/